

Toward Real-Time VLAs: Stage-Aware Two-Step Flow Denoising and System-Level Evaluation

Di Wu^{1,3} Rongtian Shen¹ Ping Liu¹ Yan Shen^{1,4} Zhenhan Yin^{1,5} Shun Zuo^{1,6}

Xuhua Chen¹ He Zheng¹ Lingfeng Zhang¹ Jianglin Zhang² Tao Zhang^{1,*}

¹Magic-Lab Team, Magiclab Robotics Inc. ²Zhejiang University ³Southeast University

⁴Harbin Institute of Technology ⁵Tongji University ⁶Jilin University

Abstract

Vision-language-action (VLA) models face a timing gap between low-rate inference and high-rate robot execution. We characterize this gap through end-to-end latency measurements of model inference and the robot execution chain. Repeated Flow Matching denoising contributes substantially to inference cost, while robot-side delays mainly arise from perception acquisition, communication scheduling, and physical response. Analysis of the velocity field shows relatively stable magnitude and direction in early integration, followed by stronger directional correction near the terminal steps. Based on this stage heterogeneity, we propose **two-stage non-uniform denoising**, reducing the number of steps from 10 to 2 and model-inference time from 61.557 ms to 21.956 ms. We also develop a **distributed real-time VLA framework** with independent inference, action-publication, and robot-control rates, modular observation acquisition, and action-provenance logging. Using $\pi_{0.5}$ as the baseline, we evaluate six real-time execution methods on a long-horizon physical garment-folding task. Legato performs best overall among training-based methods, while Temporal Smoothing leads among training-free methods; both perform strongly in task success, completion time, action continuity, and acceleration smoothness. Combining two-step denoising with representative execution methods substantially reduces inference cost with a small reduction in task performance. These results motivate joint optimization of model-inference efficiency and robot-system timing.

Project page: <https://embodied.magiclab.top/works/inference/index.html>

GitHub: <https://github.com/MagiclabRobotics/Inference>

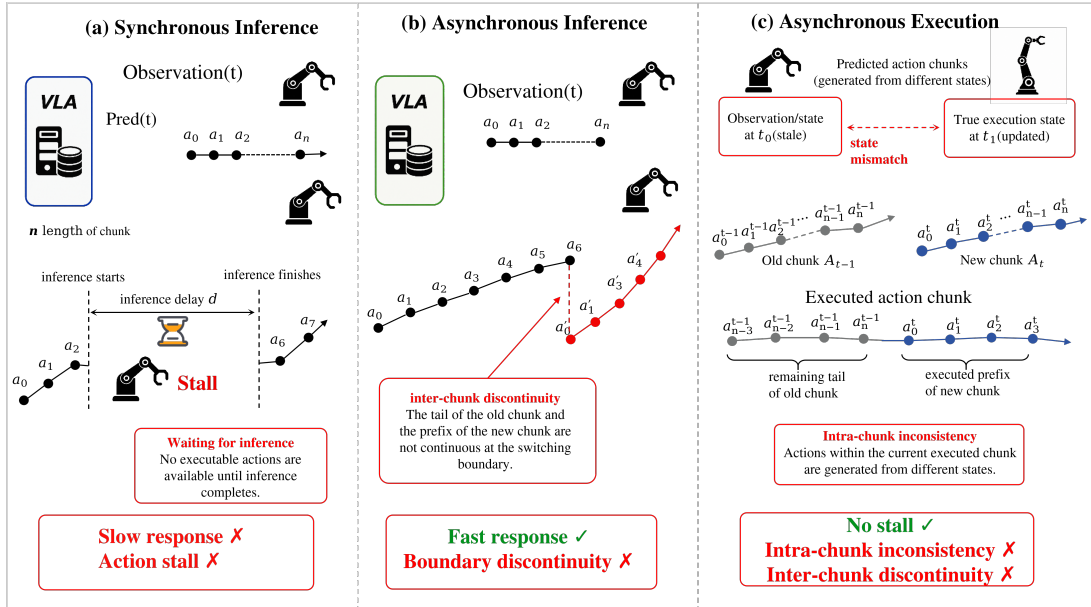


Figure 1. Time-scale mismatch between VLA inference and robot control.

*Corresponding author and project lead.

Contents

1	Introduction	3
2	Sources of End-to-End Latency and Model-Inference Bottlenecks	4
2.1	End-to-End Timeline and Non-Model Latency	4
2.2	Model-Inference Time Decomposition	5
3	Two-Stage Non-Uniform Denoising	6
3.1	Stage Heterogeneity of the Denoising Velocity Field	6
3.2	Flow Matching Action Generation and Standard Sampling	6
3.3	Two-Stage Non-Uniform Partition	7
3.4	SnapFlow Terminal Refinement and Joint Training	7
4	Distributed Real-Time Inference and Execution Framework	9
4.1	Overall Architecture	9
4.2	Pluggable Strategy Integration and Runtime Tracing	9
5	Experimental Evaluation and Analysis	10
5.1	Distributed Inference and Execution Experiments	10
5.1.1	Comparison Methods and Runtime Configurations	10
5.1.2	Experimental Platform and Task Setup	10
5.1.3	Evaluation Metrics	11
5.1.4	Comparative Analysis	12
5.1.5	Action-Chunk Continuity	12
5.1.6	Local Action-Chunk Trajectories	13
5.2	Experimental Evaluation of Two-Stage 2-NFE Denoising	14
5.2.1	Experimental Analysis	14
5.2.2	Combination with Real-Time Execution Mechanisms	14
6	Literature Review	15
6.1	Asynchronous Inference and Execution-Time Alignment	15
6.2	Inter-Chunk Continuity and Action Priors	16
6.3	Adaptive Execution Horizons and Multi-Rate Control	17
6.4	Fast Action Generation and Streaming Inference	17
6.5	Continuous-Time Action Representations and Physical Executability	18
6.6	Execution-Time Correction and System-Level Deployment	18
7	Conclusion	19
	References	20
	Appendix	24
A	Task Performance Across Garment Conditions	24
B	Training Configuration	24
C	Scale Consistency of the Flow Matching Velocity Field	25
C.1	Scale Comparison with Flow Matching GT Velocity	25
D	Offline Action Accuracy of Two-Stage Non-Uniform Denoising	25
D.1	Comparison and Evaluation Protocol	25
D.2	Per-Joint Error and Frame-Wise Variation	26
D.3	Fourteen-Dimensional Per-Joint Action Trajectories	26
D.4	Bimanual TCP Trajectories and Pose Errors	28
E	Additional Execution Figures	29

1 Introduction

Vision–language–action (VLA) models jointly model visual observations, language instructions, and robot actions, providing a path toward general-purpose robot policies with cross-task generalization [2, 20, 39, 54]. However, their computational complexity creates a substantial time-scale mismatch with the high-rate continuous control required by physical robots. Under the action-chunk execution paradigm [51], the policy periodically predicts a future action sequence from the current observation, while model inference, perception, communication, scheduling, and robot response all introduce delays. When policy updates cannot reflect the current robot and environment state in time, actions generated from historical observations may continue to affect an evolving system, misaligning perception, inference, and execution.

VLA systems commonly use asynchronous inference and execution to reduce the control pauses introduced by synchronous inference: the robot executes the current action chunk while the next is generated in parallel. As shown in Fig. 1, this overlaps computation and execution but does not eliminate temporal misalignment caused by low-rate model updates. A new chunk is conditioned on the observation available when inference starts, yet the robot state has changed by the time it takes control, potentially causing inter-chunk discontinuity. Meanwhile, the robot continues executing actions predicted from historical observations during generation, causing intra-chunk inconsistency between current perception and ongoing execution [43]. Camera exposure and readout, proprioceptive feedback, communication and task scheduling, and physical response introduce additional system-level timing offsets [46]. Real-time VLA is therefore an end-to-end timing problem jointly determined by model computation and the robot execution chain.

Existing real-time VLA research addresses these problems at three levels. The first improves model-side action-generation efficiency through distillation, fewer sampling steps, or modified sampling procedures for diffusion and Flow Matching policies [27, 28]. Low-dimensional action parameterizations, such as spline control points, further reduce generation complexity while introducing smoothness and motion-feasibility priors [14, 41, 47, 52]. The second addresses action continuity and state alignment during asynchronous execution. Executed or committed action prefixes condition generation during inference or training to reduce mode changes and boundary discontinuities [3, 4, 25]. Other methods revise actions using recent observations or predict the robot state at its future execution time, reducing observation–action and prediction–execution mismatch [37, 40, 43]. The third jointly optimizes perception, computation, and execution through sensor timing calibration, multimodal synchronization, trajectory retiming, and local tracking control [46].

Despite these advances, three issues remain. First, studies use different base models, robot platforms, tasks, inference hardware, and runtime configurations, making results difficult to compare in the absence of a unified deployment framework and evaluation standard. Second, the roles of different integration stages in the widely used Flow Matching action-generation process remain insufficiently characterized. Third, studies commonly optimize individual components without unified end-to-end latency measurement covering model inference, perception, and robot response, making it difficult to distinguish the effects of different latency sources.

Using $\pi_{0.5}$ [35] as the baseline, we first measure latency at both the model and robot levels to identify the main end-to-end sources. We then analyze visual–language conditioning and Flow Matching generation costs, together with velocity-field magnitude and direction across integration steps. These observations motivate a two-stage non-uniform denoising strategy that reduces the number of function evaluations (NFE) from 10 to 2 through non-uniform integration times, substantially improving model-inference efficiency. For system-side latency, we develop a distributed, thread-decoupled VLA inference and execution framework with modular inference, action publication, temporal alignment, and local control. Using common training data, a shared robot platform, and standardized runtime configurations, we evaluate Naive Asynchronous Execution, Temporal Smoothing, Inference-time RTC, Training-time RTC, Legato, and VLASH on a long-horizon bimanual garment-folding task. We also evaluate the proposed two-stage strategy in combination with representative real-time execution methods.

Our main contributions are:

1. **A systematic procedure for measuring model-side and robot-side latency.** Using $\pi_{0.5}$, we analyze visual–language conditioning and Flow Matching generation, and measure camera timestamp offset, image readout, proprioceptive feedback, and control response to quantify end-to-end VLA timing.
2. **A two-stage non-uniform Flow Matching denoising strategy.** Stepwise analysis reveals stage heterogeneity in Flow Matching denoising. Non-uniform integration combines standard Flow long-interval generation with SnapFlow short-interval terminal refinement, reducing NFE from 10 to 2 and mean model-inference time from 61.557 ms to 21.956 ms, a $2.804\times$ speedup.

3. **An open-source distributed real-time VLA inference and execution framework.** Modular, thread-decoupled components jointly manage observation acquisition, policy inference, action buffering, temporal alignment, local control, and runtime logging. Independent inference, action-publication, and robot-control rates provide a unified, traceable basis for deploying and evaluating real-time VLA methods.

2 Sources of End-to-End Latency and Model-Inference Bottlenecks

Real-time VLA methods improve action generation, asynchronous scheduling, action-chunk handover, and execution-time correction. However, model inference rate alone cannot explain the temporal offset accumulated before an action physically takes effect [46]. We distinguish two classes of latency: delays in perception, communication, scheduling, and physical execution; and visual–language computation and Flow Matching action generation within the model.

Table 1. Measured latency components.

Layer	Latency component	Measured value	Interpretation
Perception	Primary-camera timestamp offset	17.7 ± 1.8 ms	Offset of the RealSense internal timestamp relative to actual image acquisition
Perception	Primary-camera image readout	32.8 ± 3.3 ms	Input/output (I/O) delay from the RealSense timestamp to image receipt by the host process
State	Proprioceptive feedback	26.4 ± 6.2 ms	Delay from the physical robot state to host observation of the corresponding software development kit (SDK) feedback
Execution	Command-to-motion response	15.3 ± 5.4 ms	Delay from command publication until physical motion follows the target phase
Model	Standard Flow, 10 NFE	61.6 ± 6.2 ms	Latency statistics from 620 inference calls

2.1 End-to-End Timeline and Non-Model Latency

A closed-loop update begins with image formation and a change in robot state, followed by image readout, state feedback, input preprocessing, request transmission, policy inference, result transmission, action buffering and publication, and finally physical response. The interval from host receipt of an observation to availability of a new action chunk is

$$T_{\text{avail}} = T_{\text{pre}} + T_{\text{up}} + T_{\text{model}} + T_{\text{down}} + T_{\text{queue}}, \quad (1)$$

where T_{pre} denotes input preprocessing, T_{up} and T_{down} denote request upload and action-chunk download, and T_{queue} includes service queueing, action buffering, and control-period quantization. Equation (1) excludes perception latency accumulated before host receipt and physical response after command publication. The visual information used when an action takes effect is therefore also subject to the camera timestamp offset t_{camera} and image-readout latency t_{readout} . Proprioceptive state carries feedback latency t_{proprio} , and the output path includes motion-response latency t_{motion} .

We measure these components using the joint calibration apparatus in Fig. 2. The robot executes small-amplitude periodic sinusoidal joint motions while a display presents a host time code and phase bar. A RealSense camera records both the visual time code and an end-effector ArUco marker; the host logs camera timestamps, image receipt times, joint commands, and SDK feedback. The difference between host receipt time and the RealSense frame timestamp estimates image-readout latency, while the display time code recovers the actual image-acquisition time. Phase or correlation relationships among periodic commands, visual marker trajectories, and state feedback separate motion-response latency from proprioceptive-feedback latency. Calibration uses visual acquisition at 30 frames/s and state recording at 100 Hz, with repeated measurements across joints and motion frequencies.

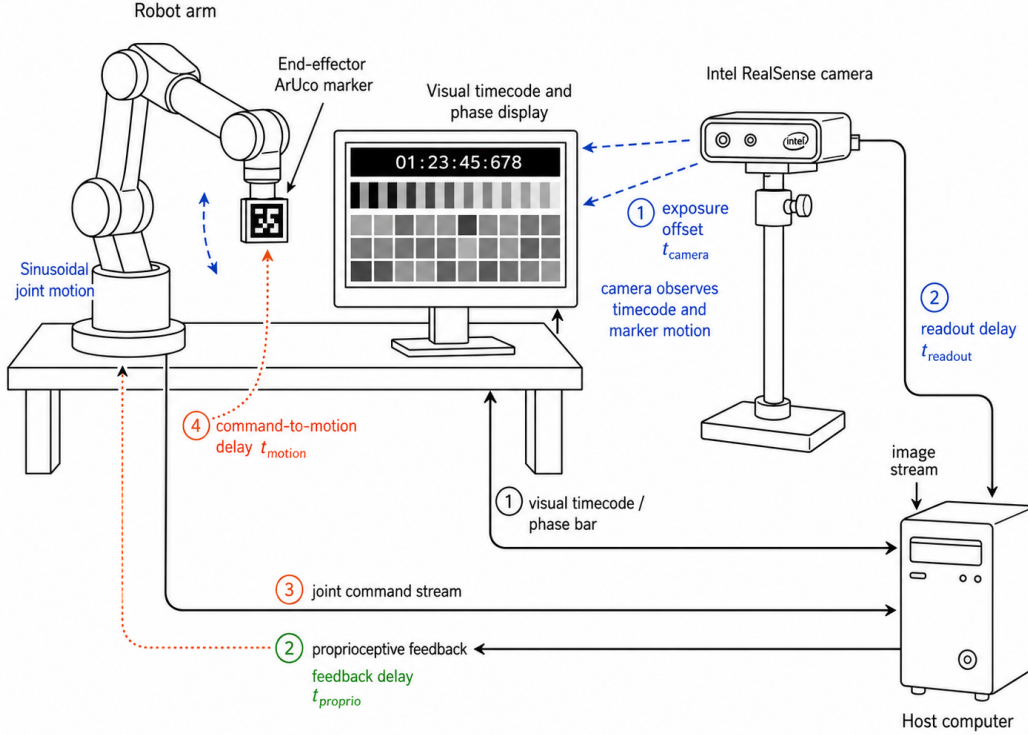


Figure 2. Joint system-latency calibration apparatus and signal-acquisition workflow. The RealSense camera records a visual time code on the display and an end-effector ArUco marker, while the host logs image timestamps, receipt times, joint commands, and proprioceptive feedback. Timestamp differences and phase relationships among periodic signals estimate the camera timestamp offset, image-readout latency, motion-response latency, and proprioceptive-feedback latency.

Table 1 summarizes the principal latency components measured on the physical robot system. Beyond model inference, perception, state feedback, and physical execution each incur non-negligible delays. The primary-camera timestamp offset and image-readout latency are 17.690 ms and 32.817 ms, respectively; proprioceptive-feedback latency is 26.4 ± 6.2 ms, and command-to-motion response latency is 15.3 ± 5.4 ms. Thus, the observations available to the controller are inherently misaligned in time with the physical robot state. Faster model inference alone cannot eliminate this mismatch.

2.2 Model-Inference Time Decomposition

For a VLA with a Flow Matching action head, one sampling call can be expressed as

$$T_{\text{model}}(N) = T_{\text{cond}} + \sum_{k=1}^N T_{\text{step}}^{(k)} + T_{\text{post}}, \quad (2)$$

where T_{cond} includes image preprocessing, visual–language prefix encoding, and condition caching, whose cost does not scale proportionally with the number of denoising steps. $T_{\text{step}}^{(k)}$ is the cost of the k th Flow Matching sampling step, including action-head evaluation and its numerical integration update; T_{post} covers action denormalization and output transformation. Model-inference time therefore comprises fixed costs independent of NFE and repeated sampling costs that increase with NFE.

On an NVIDIA GeForce RTX 4090 D (24 GB) in JAX, standard 10-NFE Flow takes 61.557 ms on average across 620 inference calls after excluding the first just-in-time (JIT) compilation. This result indicates substantial model-side computation in iterative Flow sampling. The next section analyzes differences between sampling stages and introduces two-stage non-uniform time integration to reduce the NFE required for action generation.

3 Two-Stage Non-Uniform Denoising

The measurements in Section 2 show that, in addition to non-model delays in perception, state feedback, and physical execution, standard Flow action generation requires repeated velocity-field evaluations. Reducing the number of function evaluations during action generation is therefore a direct route to lower model-inference cost. Simply reducing the number of integration steps, however, generally increases discretization error. The key question is how to allocate a limited computational budget along Flow time.

3.1 Stage Heterogeneity of the Denoising Velocity Field

To characterize changes between consecutive Flow Matching iterations, we fix model parameters, language condition, and initial noise, and record complete velocity-field trajectories from standard 10-NFE inference for different observations. For the velocity field $v_k \in \mathbb{R}^{50 \times 14}$ at evaluation k , root mean square (RMS) measures its overall magnitude in normalized action space:

$$r_k = \frac{\|v_k\|_F}{\sqrt{50 \times 14}}, \quad (3)$$

where $\|\cdot\|_F$ is the Frobenius norm. The angle between adjacent velocity fields measures the directional correction at integration step k :

$$\theta_k = \arccos\left(\frac{\langle v_k, v_{k-1} \rangle_F}{\|v_k\|_F \|v_{k-1}\|_F}\right) \frac{180^\circ}{\pi}, \quad k = 2, \dots, 10. \quad (4)$$

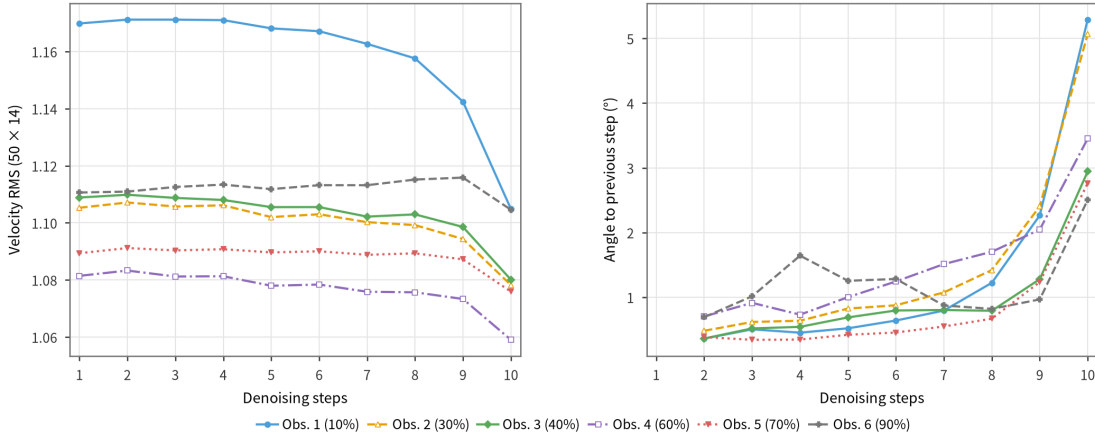


Figure 3. Velocity-field trajectories under standard 10-NFE Flow sampling. Changes in velocity RMS r_k (left) and the angle θ_k between adjacent integration steps (right) are concentrated near the endpoint.

As shown in Fig. 3, all six velocity RMS trajectories remain broadly stable during the early and intermediate stages, with small angles between adjacent velocity fields. Near the denoising endpoint, most trajectories show more pronounced magnitude changes, and all six angle trajectories rise at the final step. These observations indicate that network evaluations contribute unevenly to action generation: early and intermediate updates progress along similar velocity fields, whereas terminal updates concentrate magnitude adjustments and directional corrections. This stage difference motivates long-interval coarse generation followed by short-interval terminal refinement.

Accordingly, we allocate the two network evaluations non-uniformly along Flow time. The first spans a longer interval to generate the main action chunk, and the second refines the prediction over a short interval near the endpoint.

3.2 Flow Matching Action Generation and Standard Sampling

Let c denote the condition formed by three images, a language instruction, and robot state. Let $x_t \in \mathbb{R}^{H \times d}$ be the normalized action chunk at time t , where $t = 1$ denotes Gaussian noise and $t = 0$ denotes the target action. The Flow Matching [23] action expert learns the conditional velocity field

$$\frac{dx_t}{dt} = v_\theta(x_t, t; c), \quad x_1 \sim \mathcal{N}(0, I). \quad (5)$$

For a decreasing time grid $1 = t_0 > t_1 > \dots > t_N = 0$, standard Euler sampling applies

$$x_{t_{k+1}} = x_{t_k} + (t_{k+1} - t_k)v_\theta(x_{t_k}, t_k; c). \quad (6)$$

The action expert therefore performs N function evaluations. Section 2 shows that this repeated path grows with NFE and is a direct target for reducing model-side cost.

3.3 Two-Stage Non-Uniform Partition

We partition the integration interval into $[1, \tau]$ and $[\tau, 0]$, fixing $\tau = 0.3$ in the current implementation. The first stage spans an interval of length 0.7 for coarse action generation. The second spans the final 0.3 interval to restore action details and correct terminal residuals.

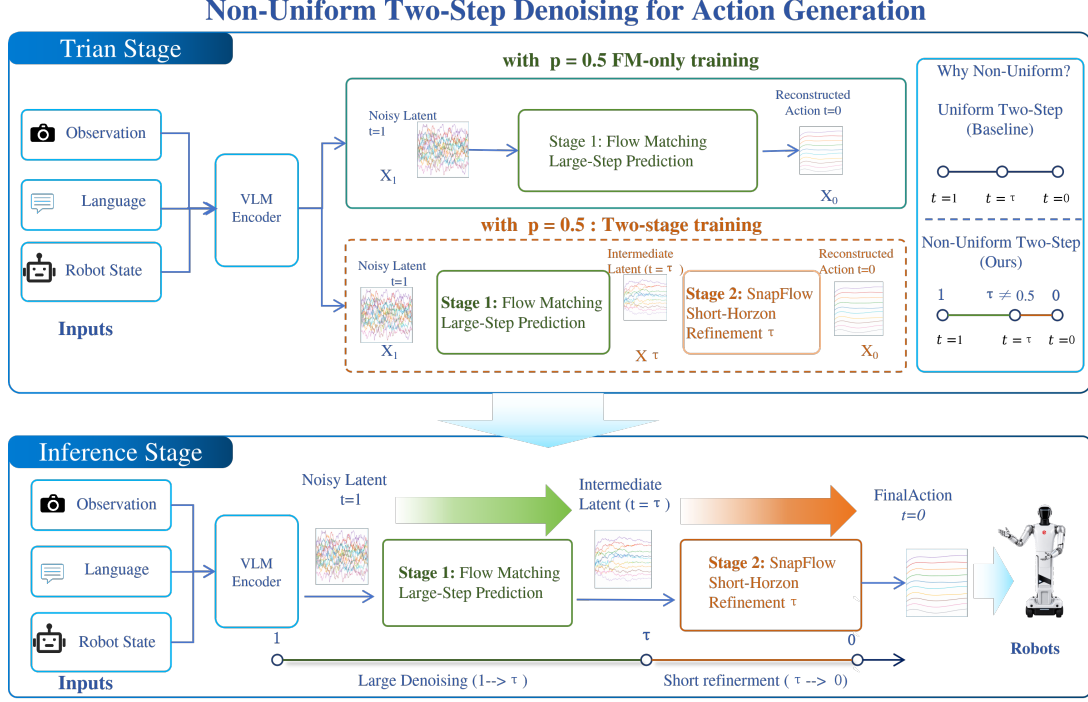


Figure 4. Training and inference pipeline for two-stage non-uniform action denoising. Training selects the standard Flow Matching branch or the two-stage branch with equal probability. The latter applies a large Flow update followed by short-interval mean-velocity prediction for terminal refinement. Inference reuses the same visual–language–proprioceptive condition and evaluates the action expert only twice along the non-uniform schedule $1 \rightarrow \tau \rightarrow 0$. The right side compares uniform two-step sampling with the proposed asymmetric schedule.

As shown in Fig. 4, the first stage evaluates a local Flow velocity at $t = 1$ and applies a large Euler step. The second takes the predicted state x_τ , current time τ , and target time 0 as inputs to predict the mean velocity over the terminal interval. Both velocities are predicted by the same action expert and distinguished by target time s . Define $\mathcal{V}_\theta(x, t \rightarrow s; c) = \text{clip}(v_\theta(x, t, s; c), -C, C)$, where $C = 20$. Here, $s = t$ denotes local velocity prediction, and $s = 0$ denotes cross-interval velocity prediction directly to the denoising endpoint. The updates are

$$\begin{aligned} v^{(1)} &= \mathcal{V}_\theta(x_1, 1 \rightarrow 1; c), & x_\tau &= x_1 - (1 - \tau)v^{(1)}, \\ \bar{v}^{(2)} &= \mathcal{V}_\theta(x_\tau, \tau \rightarrow 0; c), & x_0 &= x_\tau - \tau\bar{v}^{(2)}, \end{aligned} \quad (7)$$

with no teacher branch during deployment and only two action-expert evaluations. The stages share condition c and action-expert parameters, but each still executes a complete stage computation. A fivefold reduction in NFE therefore does not imply a fivefold reduction in execution time.

3.4 SnapFlow Terminal Refinement and Joint Training

In the proposed two-stage framework, the first stage uses standard Flow Matching for long-interval action generation. The second directly adopts the shortcut refinement mechanism of SnapFlow [28], restricted to the short terminal

interval $[\tau, 0]$. Unlike one-step or few-step compression over the full Flow interval, our method retains standard Flow generation in the first stage and confines SnapFlow to the terminal interval.

Algorithm 1 Two-stage non-uniform denoising: training and deployment inference.

Require: Condition c , training action chunk $x_0, \tau = 0.3, \lambda_{sc} = 0.1$

Ensure: Updated parameters θ ; generated action chunk $\hat{x}_0$ at deployment

Training:

- ```

1: Sample $x_1 \sim \mathcal{N}(0, I)$; set $v \leftarrow x_1 - x_0$
2: Sample $b \in \{0, 1\}$ with equal probability
3: if $b = 1$ then ▷ Standard Flow Matching
4: Sample $t \sim \mathcal{U}(0, 1)$ for each example
5: $x_t \leftarrow (1 - t)x_0 + tx_1$
6: $\hat{v} \leftarrow \mathcal{V}_\theta(x_t, t \rightarrow t; c)$
7: $\mathcal{L} \leftarrow \text{MSE}(\hat{v}, v^*)$
8: else ▷ Non-uniform two-stage branch
 Stage 1: Standard Flow large-step prediction
9: $v^{(1)} \leftarrow \mathcal{V}_\theta(x_1, 1 \rightarrow 1; c)$
10: $x_\tau \leftarrow x_1 - (1 - \tau)v^{(1)}$
 Stage 2: SnapFlow short-horizon refinement
11: $\tilde{x}_\tau \leftarrow \text{sg}(x_\tau)$
12: $v_a \leftarrow \text{sg}(\mathcal{V}_\theta(\tilde{x}_\tau, \tau \rightarrow \tau; c))$
13: $x_{\tau/2} \leftarrow \tilde{x}_\tau - \frac{\tau}{2}v_a$
14: $v_b \leftarrow \text{sg}(\mathcal{V}_\theta(x_{\tau/2}, \tau/2 \rightarrow \tau/2; c))$
15: $\bar{v}_{\tau \rightarrow 0}^* \leftarrow (v_a + v_b)/2$
16: $\bar{v}^{(2)} \leftarrow \mathcal{V}_\theta(\tilde{x}_\tau, \tau \rightarrow 0; c)$
17: $\mathcal{L} \leftarrow \text{MSE}(v^{(1)}, v^*)$
 $\quad + \lambda_{\text{sc}} \text{MSE}(\bar{v}^{(2)}, \bar{v}_{\tau \rightarrow 0}^*)$
18: end if
19: Update parameters θ using $\nabla_\theta \mathcal{L}$
Inference:
20: Sample $x_1 \sim \mathcal{N}(0, I)$
21: $x_\tau \leftarrow x_1 - (1 - \tau)\mathcal{V}_\theta(x_1, 1 \rightarrow 1; c)$
22: $\hat{x}_0 \leftarrow x_\tau - \tau\mathcal{V}_\theta(x_\tau, \tau \rightarrow 0; c)$
23: return $\hat{x}_0$ ▷ Two action-expert evaluations

```

The second stage follows SnapFlow in constructing cross-interval mean-velocity supervision from a shortcut target, an idea originating in Shortcut Models [12]. Starting from the actual first-stage prediction  $x_\tau$ , we apply stop-gradient to obtain  $\tilde{x}_\tau = \text{sg}(x_\tau)$ , preventing the terminal refinement loss from backpropagating through the first-stage long-interval prediction. Two local velocity evaluations within  $[\tau, 0]$  construct the supervision target:

$$\begin{aligned} v_a &= \text{sg}(\mathcal{V}_\theta(\tilde{x}_\tau, \tau \rightarrow \tau; c)), & x_{\tau/2} &= \tilde{x}_\tau - \frac{\tau}{2}v_a, \\ v_b &= \text{sg}(\mathcal{V}_\theta(x_{\tau/2}, \tau/2 \rightarrow \tau/2; c)), & \bar{v}_{\tau \rightarrow 0}^* &= \frac{v_a + v_b}{2}. \end{aligned} \quad (8)$$

Here,  $\text{sg}(\cdot)$  denotes stop-gradient, and  $\bar{v}_{\tau \rightarrow 0}^*$  is the terminal-interval mean-velocity target constructed by the SnapFlow teacher path. The second-stage student takes  $\tilde{x}_\tau$  as input and directly predicts the mean velocity from time  $\tau$  to endpoint 0:

$$\bar{v}^{(2)} = \mathcal{V}_\theta(\tilde{x}_\tau, \tau \rightarrow 0; c). \quad (9)$$

To retain both standard Flow Matching local-velocity modeling and SnapFlow cross-interval prediction, training selects the standard Flow Matching branch or the two-stage branch with equal probability at the batch level. For the latter, the joint objective is

$$\mathcal{L}_{2\text{stage}} = \text{MSE}\left(v^{(1)}, v^{(1)*}\right) + \lambda_{\text{sc}} \text{MSE}\left(\bar{v}^{(2)}, \bar{v}_{\tau \rightarrow 0}^*\right), \quad (10)$$

where MSE averages the squared error over action-chunk elements. The first-stage loss weight is fixed at 1.0, and the SnapFlow terminal refinement weight is  $\lambda_{sc} = 0.1$ .

For a training action chunk  $x_0$  and Gaussian noise  $x_1$ , standard Flow Matching uses the linear probability path

$$x_t = (1 - t)x_0 + tx_1, \quad (11)$$

with local velocity target  $v^* = x_1 - x_0$ . Thus,  $v^{(1)*} = v^*$  in Eq. (10).

Our design does not redefine SnapFlow shortcut supervision. Instead, motivated by the observed heterogeneity across Flow integration stages, it combines standard Flow long-interval generation with SnapFlow short-interval terminal refinement to form the non-uniform schedule  $1 \rightarrow \tau \rightarrow 0$ , with  $\tau = 0.3$  in the current implementation. Algorithm 1 summarizes training and deployment inference.

## 4 Distributed Real-Time Inference and Execution Framework

### 4.1 Overall Architecture

To address the system-side latency analyzed in Section 2, we decouple robot-side observation acquisition and high-rate control from GPU-side policy computation, as shown in Fig. 5. The robot-side Collector/RobotIO aggregates the overhead camera, two wrist cameras, and robot state, continuously caching the latest observation. At a configured rate, the inference thread reads a snapshot and assembles images, a language instruction, robot state, sampling steps, and execution-mode-specific latency fields into a request. Cross-host deployment uses a Transmission Control Protocol socket or WebSocket, whereas same-host deployment can use shared memory. The GPU policy server parses requests, transforms inputs, performs VLA inference, and returns action chunks with latency metadata.

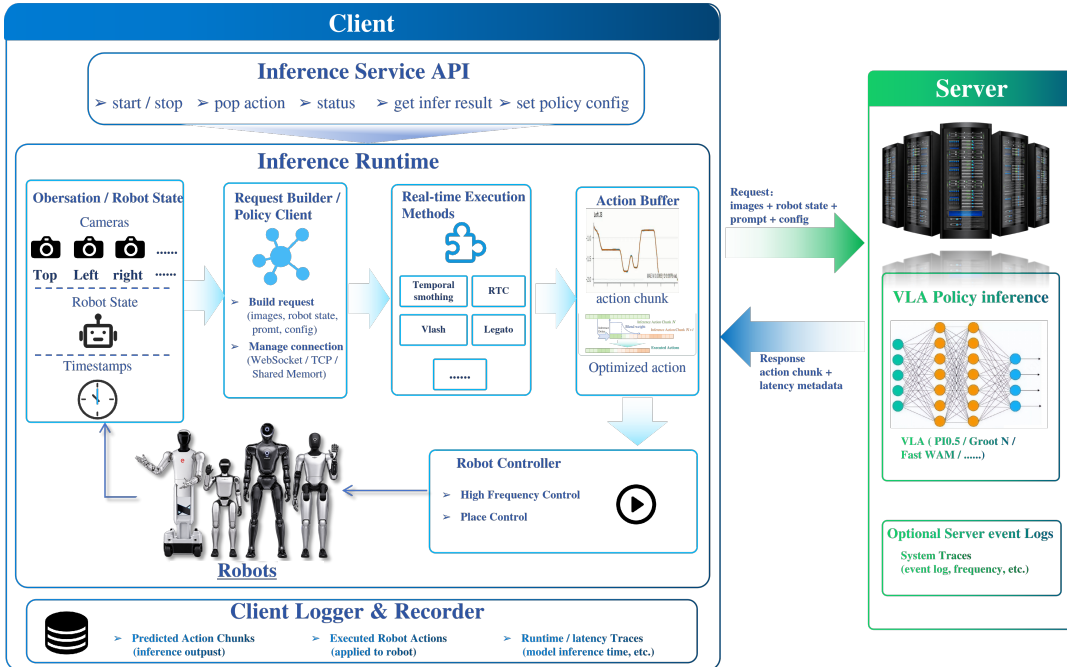


Figure 5. Distributed real-time VLA inference and execution framework.

The client uses latency estimates to determine each chunk’s expected effective index, then handles handover, blending, and indexing in the action buffer. An independent publication thread sends individual commands at the control rate, preventing low-rate inference and communication waits from blocking high-rate control. The framework also records sensor timestamps and action provenance. Hardware-specific clock semantics, fixed offsets, and wrist-camera timing differences require separate calibration.

### 4.2 Pluggable Strategy Integration and Runtime Tracing

The mode manager encapsulates synchronous execution and the six real-time VLA strategies in Table 2 as pluggable modules that share observation, communication, buffering, and control paths. Logs record request submission and return, model inputs and outputs, action-chunk indices, published actions and their provenance, and request, inference, and transmission times. These records support reconstruction of action handovers and localization of errors by system layer. The experiments demonstrate unified integration, physical robot execution, and trajectory traceability for all six methods.

**Table 2. Layered taxonomy of six real-time VLA execution strategies.**

| Method             | System layer          | Principal operation                                                | Primary objective                                  |
|--------------------|-----------------------|--------------------------------------------------------------------|----------------------------------------------------|
| Naive Asynchronous | Scheduling            | Decouple model inference from robot execution                      | Remove robot waiting during inference              |
| Temporal Smoothing | Action buffer         | Blend the old-chunk tail with the new-chunk prefix                 | Suppress velocity and acceleration discontinuities |
| Inference-time RTC | Generation constraint | Fix committed actions and complete the remaining trajectory        | Reduce mismatch between old and new chunks         |
| Training-time RTC  | Policy conditioning   | Simulate latency and condition on an action prefix during training | Learn low-overhead action continuation             |
| Legato             | Learned generation    | Learn native action-continuation dynamics                          | Internalize inter-chunk continuity                 |
| VLASH              | State alignment       | Propagate state to the expected execution time                     | Reduce prediction error caused by stale state      |

## 5 Experimental Evaluation and Analysis

This section evaluates the distributed real-time execution framework and the two-stage 2-NFE denoising strategy. Execution-layer experiments deploy six real-time VLA methods in a unified framework and use physical garment-folding results and action-continuity measurements to compare task performance and execution characteristics under practical deployment configurations. Model-side experiments compare inference time and action error for standard 10-NFE and two-stage 2-NFE sampling under fixed offline inputs and a common timing protocol, quantifying the computational gains and error changes from fewer sampling steps.

### 5.1 Distributed Inference and Execution Experiments

To evaluate the framework’s support for different real-time execution methods, we deploy six representative methods under a common software and hardware setup and compare them on a long-horizon bimanual garment-folding task. The evaluation covers task completion, execution efficiency, and action continuity.

#### 5.1.1 Comparison Methods and Runtime Configurations

We evaluate Naive Asynchronous Execution, Temporal Smoothing, Inference-time RTC, Training-time RTC, Legato, and VLASH. Table 2 summarizes their system layers and principal operations.

Fig. S5 and Fig. S6 use a common action-chunk timeline to illustrate inference scheduling and handover mechanisms. Synchronous inference and Temporal Ensembling [51] serve as reference configurations for framework validation and are excluded from the unified physical comparison and its reported results.

#### 5.1.2 Experimental Platform and Task Setup

The bimanual platform comprises two Agilex Piper manipulators. Robot actions and proprioceptive state contain 12 arm-joint dimensions and two gripper dimensions. An overhead camera and two wrist cameras each acquire  $640 \times 480$  RGB images at 30 frames/s.

We use  $\pi_{0.5}$  as the common base policy and bimanual T-shirt folding as the long-horizon physical task. The three T-shirts differ in size and color: small purple, large pink, and medium red. These garments introduce variation in object appearance and scale. Each method is evaluated independently ten times on each garment, yielding 30 trials per method and 180 physical trials in total. Inference rate, action-publication rate, and Flow Matching NFE use the best configurations identified for each method in preliminary experiments. Final evaluation uses the same hardware platform and task conditions.

Each trial starts from a randomly wrinkled garment. The two arms successively grasp keypoints, flatten the garment, lay it down, and execute three folds. Fig. 8 shows the complete workflow, and Fig. 7 shows the three garment conditions. All methods use the same task definition and success criteria.



Figure 6. Bimanual robot platform used in the experiments.



Figure 7. Garments with different sizes and colors used in the experiments.

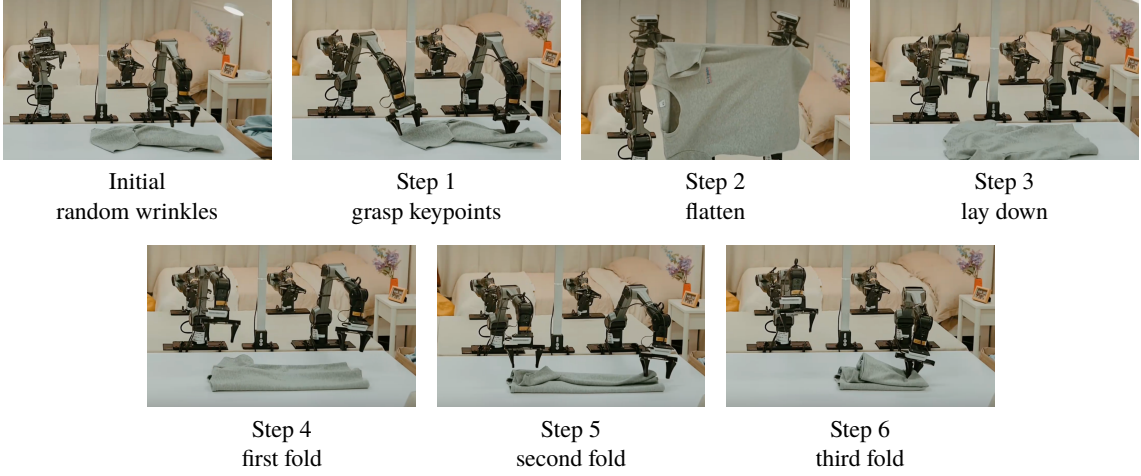


Figure 8. Stage sequence of the bimanual garment-folding task.

### 5.1.3 Evaluation Metrics

Evaluation metrics cover task performance, action-chunk continuity, system latency, and physical tracking, measuring task completion, command continuity, temporal misalignment, and reference-trajectory tracking, respectively. These metrics are not interchangeable.

**Task-level metrics** For  $N$  trials of a method, let  $S$  denote the number of successes and  $T_i$  the duration of trial  $i$ . Success rate and mean execution time are

$$R_{\text{succ}} = \frac{S}{N}, \quad \bar{T} = \frac{1}{N} \sum_{i=1}^N T_i. \quad (12)$$

To account for both success probability and execution speed, we also report successful-task throughput,

$$\mathcal{Q} = R_{\text{succ}} \frac{3600}{\bar{T}}, \quad (13)$$

in successful tasks per hour. We report a 95% Wilson confidence interval for the success rate of each method across its 30 trials. The experiments are not paired, so we do not conduct between-method significance tests.

**Action-chunk continuity and smoothness metrics** Let  $a_k \in \mathbb{R}^m$  be the action published to the robot controller at time  $k$ , and let  $\hat{a}_k^{\text{tail}}$  be the time-aligned prediction from the previous chunk at the same time. For the valid-overlap set  $\Omega$ , the instantaneous trajectory deviation is

$$d_k = \frac{1}{m} \sum_{j=1}^m |a_{k,j} - \hat{a}_{k,j}^{\text{tail}}|. \quad (14)$$

Tail Gap averages the deviation over all valid-overlap times, whereas Switch Gap uses only the handover set  $\mathcal{B}$ :

$$G_{\text{tail}} = \frac{1}{|\Omega|} \sum_{k \in \Omega} d_k, \quad G_{\text{switch}} = \frac{1}{|\mathcal{B}|} \sum_{k \in \mathcal{B}} d_k. \quad (15)$$

A low Switch Gap therefore indicates a well-connected handover instant but does not guarantee agreement over the subsequent predicted tail.

For a single joint,  $m = 1$ , and the metrics reduce to mean absolute joint-position deviations in rad. Dynamic command smoothness is evaluated using finite-difference velocity and acceleration at control period  $h$ :

$$v_k = \frac{a_k - a_{k-1}}{h}, \quad \alpha_k = \frac{a_k - 2a_{k-1} + a_{k-2}}{h^2}. \quad (16)$$

We report maximum absolute acceleration in representative trials. Tail Gap and Switch Gap characterize geometric continuation between chunks, whereas maximum acceleration measures dynamic impact at switching. The quantities are not equivalent.

#### 5.1.4 Comparative Analysis

As shown in Table 3, the six methods differ markedly on long-horizon bimanual garment folding. Legato and VLASH achieve the strongest overall task performance, completing 29/30 and 28/30 trials, respectively. Legato leads all three task metrics with 96.7% success, 73.56 s mean completion time, and 47.31  $\text{h}^{-1}$  successful-task throughput. VLASH ranks second overall, with 93.3% success and 43.22  $\text{h}^{-1}$  throughput.

Legato performs best among training-based methods, while Temporal Smoothing provides the strongest overall performance among training-free methods, with 76.7% success and 29.38  $\text{h}^{-1}$  throughput. Training-time RTC, Naive Asynchronous, and Inference-time RTC achieve success rates no higher than 63.3% and mean completion times exceeding 120 s. Notably, Inference-time RTC does not outperform the other methods despite inference at 3 Hz. A higher replanning rate alone therefore does not guarantee better task performance; the outcome also depends on the interaction between chunk continuity, state mismatch, and execution mechanisms.

Table 3. Task-level results for real-time VLA execution methods.

| Method             | Inference (Hz) | Publication (Hz) | NFE | Success rate         | 95% Wilson interval | Mean time (s) | Throughput ( $\text{h}^{-1}$ ) |
|--------------------|----------------|------------------|-----|----------------------|---------------------|---------------|--------------------------------|
| VLASH              | 2              | 30               | 10  | 28/30 (93.3%)        | [78.7, 98.2]        | 77.73         | 43.22                          |
| Training-time RTC  | 1              | 30               | 10  | 19/30 (63.3%)        | [45.5, 78.1]        | 129.37        | 17.62                          |
| Legato             | 2              | 50               | 5   | <b>29/30 (96.7%)</b> | [83.3, 99.4]        | <b>73.56</b>  | <b>47.31</b>                   |
| Temporal Smoothing | 3              | 30               | 10  | 23/30 (76.7%)        | [59.1, 88.2]        | 93.93         | 29.38                          |
| Naive Asynchronous | 1              | 30               | 10  | 19/30 (63.3%)        | [45.5, 78.1]        | 124.13        | 18.37                          |
| Inference-time RTC | 3              | 30               | 10  | 18/30 (60.0%)        | [42.3, 75.4]        | 133.73        | 16.15                          |

#### 5.1.5 Action-Chunk Continuity

We use the third left-arm joint, `left_j3`, as a representative joint to analyze action continuity. Table 4 shows clear differences in action-continuity metrics among the real-time execution methods. Legato achieves the lowest Tail Gap and Switch Gap, 0.0193 rad and 0.0062 rad, indicating reduced tail error and handover discontinuity. VLASH also performs well, with gaps of 0.0459 rad and 0.0173 rad. By contrast, Naive Asynchronous reaches a maximum acceleration of 744.1  $\text{rad/s}^2$  and has large values for both gaps, showing that direct asynchronous chunk replacement can cause pronounced trajectory discontinuities.

Table 4. Action-continuity results for `left_j3`; lower is better for every metric.

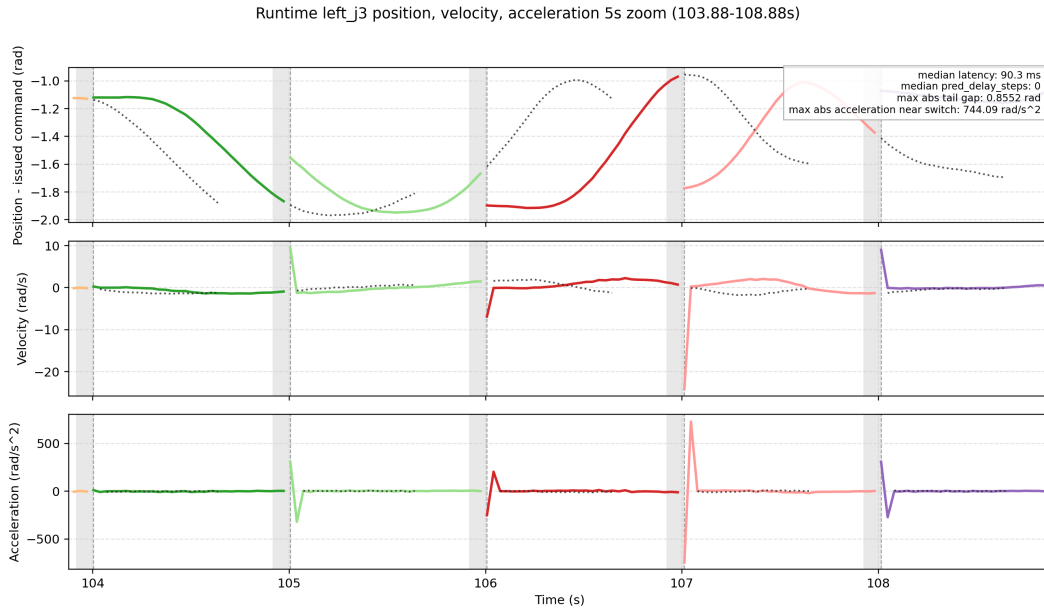
| Method             | Maximum acceleration ( $\text{rad/s}^2$ ) | Tail Gap (rad) | Switch Gap (rad) |
|--------------------|-------------------------------------------|----------------|------------------|
| Naive Asynchronous | 744.1                                     | 0.1799         | 0.1538           |
| Temporal Smoothing | <b>25.6</b>                               | 0.0967         | 0.0998           |
| Inference-time RTC | 65.55                                     | 0.1310         | 0.1164           |
| Training-time RTC  | 392.8                                     | 0.1201         | 0.0630           |
| Legato             | 46.6                                      | <b>0.0193</b>  | <b>0.0062</b>    |
| VLASH              | 73.9                                      | 0.0459         | 0.0173           |

Temporal Smoothing achieves the lowest maximum acceleration, 25.6  $\text{rad/s}^2$ , reflecting strong local smoothing capability, but its Tail Gap and Switch Gap remain substantially higher than Legato’s. Maximum acceleration primarily reflects local smoothness, whereas Tail Gap and Switch Gap more directly characterize tail deviation and

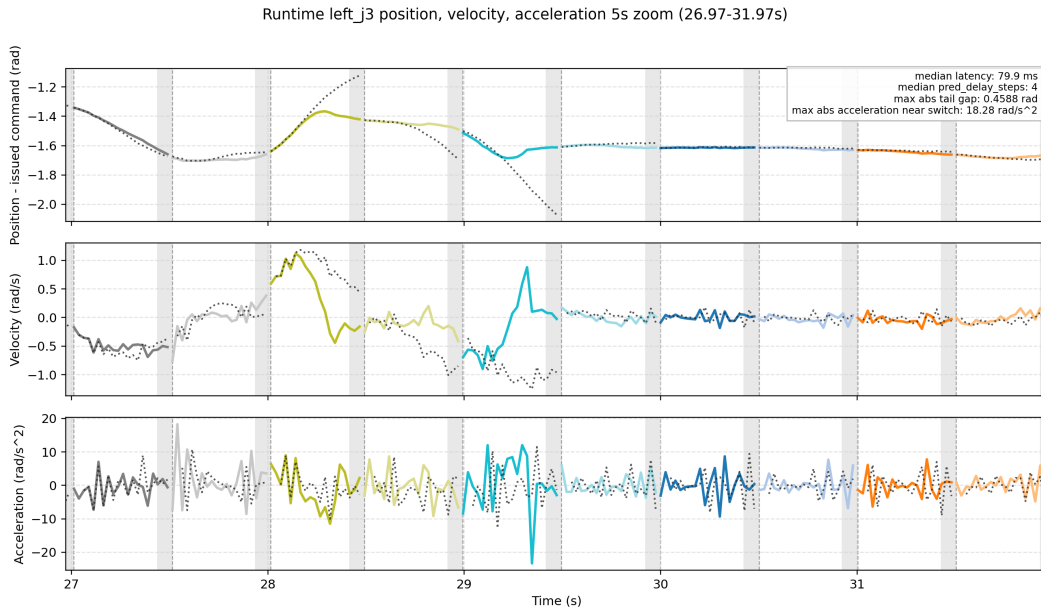
inter-chunk handover continuity. A single smoothness metric is therefore insufficient to evaluate real-time VLA execution quality.

### 5.1.6 Local Action-Chunk Trajectories

For each method, we select a 5 s window from a representative trial and plot position, velocity, and acceleration for `left_j3`. Colored solid curves indicate successive published action chunks, and black dotted curves show the time-aligned predicted tail of the previous chunk. Gray shading and vertical dashed lines mark inference intervals and chunk handovers, respectively.



**Figure 9. Local `left_j3` trajectory for Naive Asynchronous Execution.**



**Figure 10. Local `left_j3` trajectory for Legato.**

Figures 9, S7, S8, S9, S10, and 10 show the six local trajectories. Near 107 s, Naive Asynchronous jumps from  $-1.0$  rad to  $-1.8$  rad, with velocity falling to approximately  $-25$  rad/s. Maximum acceleration and maximum absolute tail deviation reach  $744.09$  rad/s<sup>2</sup> and  $0.8552$  rad, respectively. Temporal Smoothing reduces these values to  $10.27$  rad/s<sup>2</sup> and  $0.6751$  rad, primarily suppressing instantaneous impact.

Inference-time RTC exhibits a discontinuity near 13.7 s, with values of  $65.55 \text{ rad/s}^2$  and  $0.1813 \text{ rad}$ . Training-time RTC shows a velocity reset near 84–87 s, with values of  $197.16 \text{ rad/s}^2$  and  $0.8382 \text{ rad}$ . These events indicate that insufficient prefix or delay alignment can still produce handover impulses.

VLASH yields values of  $34.58 \text{ rad/s}^2$  and  $0.4615 \text{ rad}$ . Legato keeps velocity mostly within  $[-1, 1] \text{ rad/s}$ , with values of  $18.28 \text{ rad/s}^2$  and  $0.4588 \text{ rad}$ . Overall, suppressing switching impulses and reducing inter-chunk trajectory deviation are related but distinct objectives.

## 5.2 Experimental Evaluation of Two-Stage 2-NFE Denoising

We evaluate two-stage non-uniform denoising in terms of inference efficiency, offline action error, and combinations with execution mechanisms. Under fixed offline inputs, we compare inference time, joint-space action error, and bimanual Tool Center Point (TCP) error with standard 10-NFE Flow to quantify the computational gains and error changes from fewer sampling steps. We then combine the strategy with Legato and Temporal Smoothing to assess model efficiency, task performance, and action continuity under different real-time execution mechanisms.

### 5.2.1 Experimental Analysis

As shown in Table 5, the two-stage strategy reduces NFE from 10 to 2 and inference time from 61.557 ms to 21.956 ms. This corresponds to a  $2.804\times$  speedup and a 64.33% reduction in time.

**Table 5. Inference time for standard Flow and two-stage denoising.**

| Sampler               | NFE | Mean (ms)     | Speedup       | Time reduction |
|-----------------------|-----|---------------|---------------|----------------|
| Standard Flow         | 10  | 61.557        | $1.000\times$ | 0              |
| Two-stage non-uniform | 2   | <b>21.956</b> | $2.804\times$ | <b>64.33%</b>  |

**Table 6. Action-error comparison.**

| Data                | Model         | Joint MAE (rad) | Joint RMSE (rad) | Left TCP (mm) | Right TCP (mm) |
|---------------------|---------------|-----------------|------------------|---------------|----------------|
| Offline sequence I  | Standard Flow | 0.008525        | <b>0.012877</b>  | <b>11.396</b> | 7.676          |
|                     | Two-stage     | <b>0.008396</b> | 0.013329         | 11.493        | <b>6.461</b>   |
| Offline sequence II | Standard Flow | <b>0.006503</b> | <b>0.008931</b>  | 6.930         | <b>5.684</b>   |
|                     | Two-stage     | 0.006797        | 0.009104         | <b>4.831</b>  | 6.718          |

Table 6 summarizes joint error and bimanual TCP translation error on two fixed validation sequences. The methods trade advantages across sequences and metrics, but their errors remain at the same order of magnitude. Under the current fixed offline replay, two-stage 2-NFE sampling reduces model-inference time while retaining an action-error scale comparable to standard 10-NFE Flow. Appendix D provides detailed per-joint results.

### 5.2.2 Combination with Real-Time Execution Mechanisms

We integrate two-stage 2-NFE denoising into Legato and Temporal Smoothing and compare it with their respective 5-NFE and 10-NFE baselines. The experiments follow the task and evaluation protocol in Section 5.1.

As shown in Table 7, both combined configurations reduce inference time to 21.956 ms, yielding speedups of  $1.676\times$  and  $2.804\times$ , respectively. Relative to their baselines, success rates fall from 96.7% to 86.7% and from 76.7% to 70.0%. Mean successful-trial time increases by approximately 5% in both cases, and all three continuity metrics increase by approximately 20%. Thus, two-stage 2-NFE denoising can be directly combined with different real-time execution mechanisms and substantially reduce model-side action-inference cost. However, more aggressive NFE compression still incurs some closed-loop performance loss, reflecting a trade-off between inference efficiency and action-generation quality.

**Table 7. Combined evaluation of two-stage 2-NFE denoising with Legato and Temporal Smoothing.**  $\uparrow$  and  $\downarrow$  indicate that higher and lower values are better, respectively.

| Method                               | Config.  | Model efficiency |                                  |                             | Task performance            |                                        |
|--------------------------------------|----------|------------------|----------------------------------|-----------------------------|-----------------------------|----------------------------------------|
|                                      |          | NFE              | Inference time (ms) $\downarrow$ | Relative speedup $\uparrow$ | Success rate (%) $\uparrow$ | Successful-trial time (s) $\downarrow$ |
| Legato                               | Baseline | 5                | 36.808                           | 1.000 $\times$              | 29/30 (96.7)                | 73.56                                  |
| Legato + two-stage 2-NFE             | Combined | 2                | 21.956                           | 1.676 $\times$              | 26/30 (86.7)                | 76.13                                  |
| Temporal Smoothing                   | Baseline | 10               | 61.557                           | 1.000 $\times$              | 23/30 (76.7)                | 93.93                                  |
| Temporal Smoothing + two-stage 2-NFE | Combined | 2                | 21.956                           | 2.804 $\times$              | 21/30 (70.0)                | 98.63                                  |

| Method                               | Config.  | Execution continuity        |                               |                                                                         |
|--------------------------------------|----------|-----------------------------|-------------------------------|-------------------------------------------------------------------------|
|                                      |          | Tail Gap (rad) $\downarrow$ | Switch Gap (rad) $\downarrow$ | Representative-trial max. $ \alpha $ (rad/s <sup>2</sup> ) $\downarrow$ |
| Legato                               | Baseline | 0.0193                      | 0.0062                        | 46.6                                                                    |
| Legato + two-stage 2-NFE             | Combined | 0.0232                      | 0.0074                        | 55.9                                                                    |
| Temporal Smoothing                   | Baseline | 0.0967                      | 0.0998                        | 25.6                                                                    |
| Temporal Smoothing + two-stage 2-NFE | Combined | 0.1160                      | 0.1198                        | 30.7                                                                    |

## 6 Literature Review

As VLA models become widely adopted in robotics, the mismatch between low-rate inference and high-rate continuous control becomes a central real-time bottleneck. With action chunking, each model call typically predicts a future action sequence, while visual acquisition, state feedback, network transmission, model inference, and robot execution operate at different time scales. Real-time VLA research has therefore expanded beyond reducing inference time toward joint optimization of perception, inference, action generation, chunk transition, replanning, and robot execution.

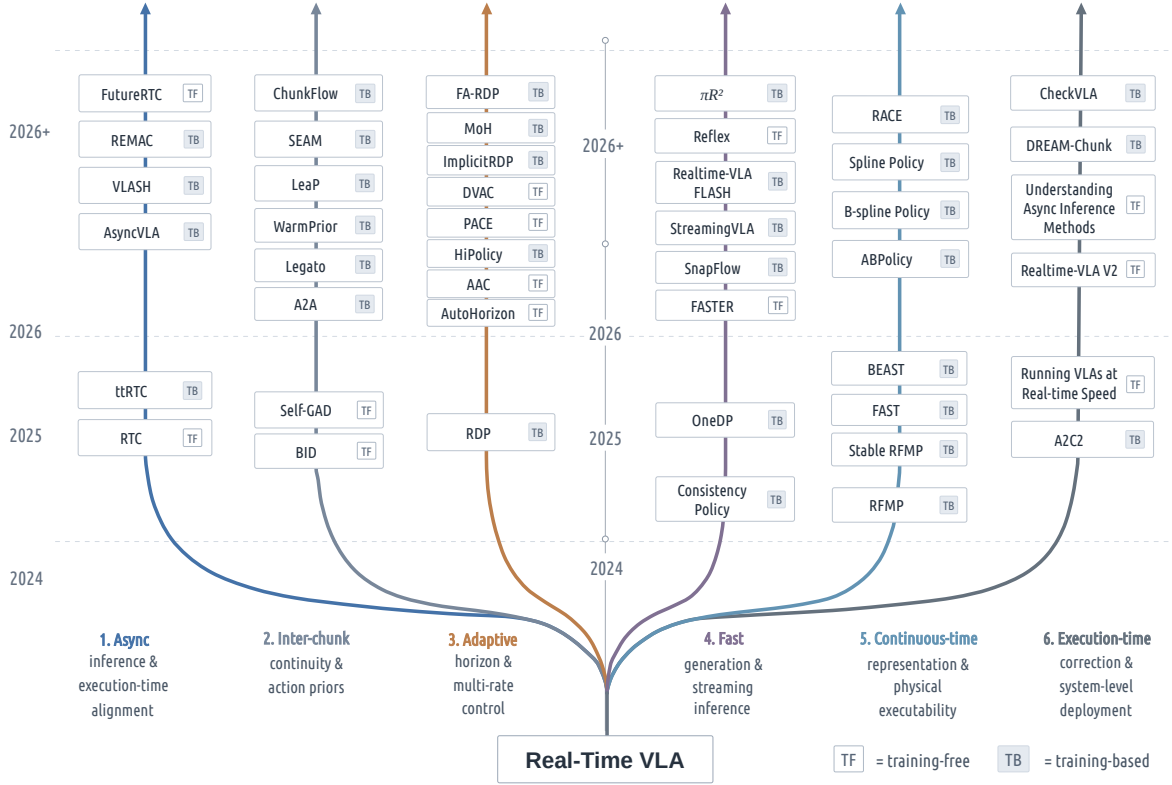
Figure 11 organizes this research by its principal technical problems. We divide the literature into six categories: (1) asynchronous inference and execution-time alignment; (2) inter-chunk continuity and action priors; (3) adaptive execution horizons and multi-rate control; (4) fast action generation and streaming inference; (5) continuous-time action representations and physical executability; and (6) execution-time correction and system-level deployment. Each study is assigned according to its primary contribution, although several methods address more than one category.

### 6.1 Asynchronous Inference and Execution-Time Alignment

Synchronous inference must wait for a new action chunk and can cause control pauses. Asynchronous execution overlaps inference with motion but separates the observation time from the new chunk’s handover time [3, 40]. Existing methods differ mainly in what they align.

Inference-time RTC and Training-time RTC treat actions that remain active during inference as a committed prefix, establishing consistent time indices between chunks. RTC fixes this prefix during Flow Matching or diffusion sampling and generates only the actions after handover, reducing conflicts between independently generated chunks [3]. Training-time RTC simulates random inference delays and committed-prefix lengths during training to learn continuation from a given prefix [4]. These methods introduce prefix constraints at inference and training time, respectively, but neither explicitly compensates for robot-state changes during inference.

Aligned action indices do not guarantee consistent conditioning states at handover. VLASH and FutureRTC therefore predict the conditions at action handover to bring model inputs closer to the future execution context. VLASH propagates proprioceptive state to the expected handover time using the actions still being executed, then conditions the next VLA prediction on that future state [40]. FutureRTC predicts both state and visual features at handover, extending alignment from proprioception to multimodal conditions [16].



**Figure 11. Chronological development and methodological taxonomy of representative real-time VLA studies.** The six branches denote asynchronous inference and execution-time alignment, inter-chunk continuity and action priors, adaptive execution horizons and multi-rate control, fast action generation and streaming inference, continuous-time action representations and physical executability, and execution-time correction and system-level deployment. Only representative studies are shown; publication dates and version information follow the cited references.

REMAC and AsyncVLA extend alignment inside the action chunk, distinguishing known states or generation progress at different positions instead of treating the chunk as an independently and synchronously generated unit. REMAC uses masked action chunking to correct and continue trajectories with partially known, missing, or perturbed actions, addressing tracking discrepancies between plans and execution [43]. AsyncVLA modifies the Flow Matching schedule so that action tokens occupy different generation stages and prioritizes low-confidence positions [17]. From prefix constraints and future-condition prediction to differentiated within-chunk generation, temporal alignment thus extends beyond the handover boundary to the asynchronous generation process.

## 6.2 Inter-Chunk Continuity and Action Priors

Consecutive diffusion or Flow Matching policy calls [8] usually start from independent random variables. Even with nearly unchanged observations, adjacent chunks can converge to different feasible modes, causing discontinuities in position, velocity, acceleration, or higher-order dynamics. The research question has evolved from smoothing a boundary to converting action history into an effective prior for the next generation process.

BID[24] uses guided test-time sampling to generate several candidate chunks. Backward consistency and forward-quality criteria select a trajectory that continues the previous decision without sacrificing future planning quality for local smoothness. Self-GAD[30] changes post-sampling selection into guidance during sampling. It constructs a prior from the previous decision and steers denoising toward temporally consistent solutions with limited extra sampling cost.

Subsequent methods incorporate continuity directly into learning or inference. Legato[25] initializes training consistently with the deployed action-continuation plan, teaching a Flow policy to extend existing trajectories. SEAM[49] uses the unexecuted tail of the previous chunk as a motion reference and analytically corrects the current Flow velocity. ChunkFlow[48] partitions actions into frozen, editable-overlap, and future regions, with explicit seam, velocity, and acceleration constraints during training. These methods move continuity modeling from positional stitching toward the generation process and higher-order kinematics.

Recent work redesigns the source distribution of generative policies. A2A[15] replaces independent Gaussian noise with a representation of action history and learns transport from executed to future actions. WarmPrior[19] shortens the Flow transport path through a temporal action prior, bringing generation into closer agreement with correlations between adjacent time points. LeaP[9] learns a state-conditioned source prior whose mean and uncertainty adapt to the robot state. Inter-chunk research is therefore shifting from smoothing generated outputs to selecting an appropriate starting point for generation. An overly strong historical prior can nevertheless propagate and amplify errors after a faulty chunk or abrupt environmental change. Online assessment of history reliability and prior strength remains necessary.

### 6.3 Adaptive Execution Horizons and Multi-Rate Control

This line of work adapts the execution horizon using system state, action structure, or model uncertainty to balance frequent inference at short horizons against action drift at long horizons.

AAC[22] adjusts the execution length from predictive uncertainty, using longer prefixes when confidence is high and requesting new observations when uncertainty increases. Without relying on internal model features, PACE[31] detects deceleration, pauses, and phase transitions in the action velocity profile as natural replanning boundaries. DVAC[11] extracts stability from Flow Matching denoising by comparing the variance of late clean-action estimates and executing only the stable, low-variance prefix. AutoHorizon[42] estimates a prediction’s valid boundary from self-attention among VLA action tokens. These approaches answer the replanning question through action uncertainty, motion phase, generation stability, and internal attention, respectively.

Beyond adapting one horizon, MoH[18] jointly models several action horizons and executes near-term actions according to cross-scale consensus. HiPolicy[50] predicts at multiple control rates. A low-rate branch provides coarse long-horizon plans, while a high-rate branch produces precise actions. Action-chunk length, execution horizon, and control rate are thus treated as coupled deployment time-scale parameters rather than fixed policy hyperparameters.

Contact-rich tasks further motivate slow–fast coordination and multimodal, multi-rate control. RDP[45] combines a low-rate visual diffusion policy with high-rate tactile or force-feedback control, preserving long-horizon visual planning capability while responding quickly to changes in contact. ImplicitRDP[6] models low-rate vision and high-rate force feedback end to end using a unified network and causal attention. FA-RDP[53] changes generation rate and sampling steps according to task phase and action multimodality. It preserves multimodal planning in free space and increases closed-loop updates during contact. These methods move real-time VLA from a fixed execution cycle toward a phase-, state-, and modality-aware adaptive control clock.

### 6.4 Fast Action Generation and Streaming Inference

Diffusion and Flow policies typically require repeated network evaluations and numerical integration to generate a complete action chunk. This line of work therefore follows two routes: reducing generation steps and changing output timing.

Consistency Policy[36] uses consistency distillation to compress a multi-step diffusion policy into one or a few steps, enabling high-rate generative robot policies. OneDP[44] applies one-step distribution distillation, generating a complete action sequence in a single forward pass. For Flow Matching policies, SnapFlow[28] uses progressive self-distillation to learn mappings across longer Flow-time intervals. It compresses iterative ODE integration into one or a few network calls. These methods primarily reduce the cost of generating a complete action chunk.

Total chunk-generation time does not directly determine closed-loop responsiveness. FASTER[27] therefore emphasizes Time to First Action. Horizon-aware scheduling completes and releases near-term actions first while refining later actions in the background.  $\pi R^2$ [33] decouples slow visual–language conditioning from fast proprioceptive conditioning. Positions within its action buffer can remain at different generation stages, continuously exposing stabilized near-term actions. These methods shift the goal from producing a complete chunk faster to obtaining the next useful action earlier.

StreamingVLA[38] further relaxes the serial observe–generate–execute loop by overlapping observation, Flow generation, and execution. Reflex[13] improves sustained inference efficiency through caching, asynchronous vision–action pipelines, and optimized operators. Realtime-VLA FLASH[32] adopts speculative inference. A lightweight draft path proposes actions quickly, while the main model verifies them and performs full inference only when needed. The resulting progression spans multi-step, one- or few-step, near-term-first, streaming, and speculative generation.

## 6.5 Continuous-Time Action Representations and Physical Executability

The fifth category reconsiders real-time performance through action representation and physical robot capability. Conventional chunks are discrete action sequences defined at a fixed sampling rate. Higher control rates require longer and higher-dimensional outputs, yet discrete points do not guarantee continuous velocity, acceleration, or curvature. Frequency-domain, spline, manifold, and dynamic structures provide more compact and physically executable representations.

FAST[34] compresses high-frequency action sequences into the frequency domain using the discrete cosine transform. Fewer action tokens can represent a complete temporal sequence, shortening autoregressive VLA outputs. BEAST[52] encodes actions with B-spline parameters, replacing per-step tokens with a fixed number of spline parameters. Continuous reconstruction then yields smoother high-frequency control. Both methods reduce output burden through high-frequency action compression and structured tokenization.

ABPolicy[47] performs Flow Matching directly in B-spline control-point space. Asynchronous inference and spline refitting connect consecutive predictions continuously. B-spline Policy[14] defines outputs as continuous-time splines, allowing one trajectory to be sampled at different control rates and rescaled in time. Spline Policy[41] treats a spline as a queryable, editable, and constrainable trajectory object. Compared with discrete chunks, these methods partly decouple control frequency from a fixed output grid and expose interfaces for velocity, acceleration, and local trajectory correction.

Another route incorporates robot geometry or dynamics. RFMP[5] defines Flow Matching on the Riemannian manifold of robot states, aligning probability paths, velocity fields, and integration with non-Euclidean action geometry. Stable RFMP[10] adds stability constraints to improve preservation of target motion distributions under perturbation. RACE[21] uses time-optimal action-chunk execution based on robot reachability and execution speed, exceeding demonstration speed while satisfying motion constraints. Real-time VLA must therefore consider both action-generation time and whether the robot can execute the generated actions at higher rates and speeds.

## 6.6 Execution-Time Correction and System-Level Deployment

The sixth category addresses online correction, execution verification, and end-to-end real-time performance after generation or execution has begun. A valid chunk can become unsuitable because of tracking error, target motion, contact slip, or new visual evidence. Optimizing only generation latency and continuity cannot ensure closed-loop robustness on a physical system.

A2C2[37] inserts a lightweight correction head between a low-rate base VLA and high-rate control. The head reads the latest observation every control cycle and applies residual corrections to the current chunk, restoring local closed-loop feedback without repeatedly invoking the full VLA. CheckVLA[26] formulates the problem as execution-time verification. An action-conditioned world model predicts expected observation changes during execution and triggers suffix repair when actual and predicted observations differ substantially. DREAM-Chunk[7] uses a latent world model to evaluate future states for several candidate chunks and selects candidates consistent with the evolving environment using actual rollout feedback.

Real-time VLA has also expanded from an algorithmic problem into a complete deployment-system problem. Running VLAs at Real-time Speed[29] jointly optimizes model computation, caching, data transfer, and streaming execution. It shows that high-rate visual processing and continuous action output require coordinated model and runtime design. Realtime-VLA V2[46] covers camera and proprioceptive calibration, action buffering, trajectory interpolation, inference services, and robot control. It emphasizes that sensor, communication, and execution latency influence physical performance alongside model latency. Understanding Asynchronous Inference Methods for Vision-Language-Action Models[1] compares several asynchronous execution schemes under common base-model, task, and latency settings. It notes that the advantages of these methods depend strongly on actual latency, execution horizon, and control settings.

Real-time VLA methods now address asynchronous execution and chunk continuity, adaptive execution horizons, faster generation, continuous-time action representations, and execution-time correction. Although they share a real-time objective, they modify different system components and are not readily comparable under one experimental setting. Adaptive-horizon methods change execution length or replanning frequency; faster generation often changes sampling or training; continuous-time methods change the action representation; and execution-time correction may introduce additional feedback or prediction modules. Combining all these approaches in one comparison would

vary model structure, action representation, training, and execution configuration simultaneously, obscuring which mechanisms cause performance differences.

Our unified physical evaluation therefore focuses on two direct problems in asynchronous action-chunk execution: inference–execution misalignment and inter-chunk continuity. We compare Naive Asynchronous, Temporal Smoothing, Inference-time RTC, Training-time RTC, Legato, and VLASH, all deployable with the same base policy, action representation, and robot interface. Their shared model and execution pipeline allow a more focused analysis of how real-time execution mechanisms affect task performance and action continuity.

## 7 Conclusion

We analyze latency at the model-computation and execution-chain levels to address the time-scale mismatch between low-rate VLA inference and high-rate continuous robot control. The results show that real-time VLA performance is jointly constrained by model computation and multiple sources of system latency.

For the model-side bottleneck, we analyze velocity-field changes across Flow Matching integration stages. The early and intermediate velocity fields remain broadly stable, whereas terminal integration exhibits stronger directional correction. The resulting two-stage non-uniform sampling strategy uses the schedule  $1 \rightarrow 0.3 \rightarrow 0$ , reducing NFE from 10 to 2 and inference time from 61.557 ms to 21.956 ms. This corresponds to a  $2.804\times$  speedup and a 64.33% reduction in time.

For system-side latency and asynchronous execution, we develop a distributed, thread-decoupled real-time inference and execution framework that jointly manages observation acquisition, model inference, action buffering, and runtime records. Policy inference, action publication, and robot control rates can be configured independently. Six representative real-time execution methods are deployed and evaluated on physical robots through this framework, demonstrating support for different execution mechanisms.

Under a common  $\pi_{0.5}$  base policy, training set, and bimanual platform, long-horizon garment-folding experiments show that Legato and Temporal Smoothing provide the strongest overall performance among training-based and training-free methods, respectively. Combining two-stage non-uniform sampling with these mechanisms yields inference speedups of  $1.676\times$  and  $2.804\times$ , respectively.

Although this study focuses on VLA, the proposed non-uniform Flow sampling approach can also be extended to Flow Matching-based world–action models. Future work will further investigate joint optimization of inference and execution scheduling through adaptive adjustment of inference rate, sampling budget, and action-handover strategies.

## References

- [1] Ayoub Agouzoul. Understanding Asynchronous Inference Methods for Vision-Language-Action Models. *arXiv preprint arXiv:2605.08168v1*, 2026. URL <https://arxiv.org/abs/2605.08168v1>.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky.  $\pi_0$ : A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.010.
- [3] Kevin Black, Manuel Galliker, and Sergey Levine. Real-Time Execution of Action Chunking Flow Policies. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 33383–33407, 2025. doi: 10.52202/085713-1122. URL [https://proceedings.neurips.cc/paper\\_files/paper/2025/file/300ccb2187dedd4edcc07f7e76d8e553-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2025/file/300ccb2187dedd4edcc07f7e76d8e553-Paper-Conference.pdf).
- [4] Kevin Black, Allen Z. Ren, Michael Equi, and Sergey Levine. Training-Time Action Conditioning for Efficient Real-Time Chunking. *arXiv preprint arXiv:2512.05964v2*, 2025. URL <https://arxiv.org/abs/2512.05964v2>.
- [5] Max Braun, Noémie Jaquier, Leonel Rozo, and Tamim Asfour. Riemannian Flow Matching Policy for Robot Motion Learning. *arXiv preprint arXiv:2403.10672v2*, 2024. URL <https://arxiv.org/abs/2403.10672v2>.
- [6] Wendi Chen, Han Xue, Yi Wang, Fangyuan Zhou, Jun Lv, Yang Jin, Shirun Tang, Chuan Wen, and Cewu Lu. ImplicitRDP: An End-to-End Visual-Force Diffusion Policy with Structural Slow-Fast Learning. *arXiv preprint arXiv:2512.10946v2*, 2026. URL <https://arxiv.org/abs/2512.10946v2>.
- [7] Wenxi Chen, Kaidi Zhang, Chi Lin, Zhiyuan Zhang, Yu She, Yuejiang Liu, Raymond A. Yeh, Shaoshuai Mou, and Yan Gu. DREAM-Chunk: Reactive Action Chunking with Latent World Model. *arXiv preprint arXiv:2606.18589v1*, 2026. URL <https://arxiv.org/abs/2606.18589v1>.
- [8] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.026.
- [9] Meipo Dai, Qiyuan Zhuang, He-Yang Xu, Ying-Jie Shuai, Yijun Wang, Qi Dou, and Xiu-Shen Wei. Where Should Action Generation Begin? A Learnable Source Prior for Generative Robot Policies. *arXiv preprint arXiv:2606.17408v1*, 2026. URL <https://arxiv.org/abs/2606.17408v1>.
- [10] Haoran Ding, Noémie Jaquier, Jan Peters, and Leonel Rozo. Fast and Robust Visuomotor Riemannian Flow Matching Policy. *arXiv preprint arXiv:2412.10855v3*, 2025. URL <https://arxiv.org/abs/2412.10855v3>.
- [11] Xiangdong Feng, Yuxuan Cheng, Chen Shi, Boyao Han, Yuxuan Yan, Yitong Hong, Zhuotao Tian, and Li Jiang. Denoising Tells When to Replan: Denoising-Variance Adaptive Chunking for Flow-Based Robot Policies. *arXiv preprint arXiv:2606.03847v1*, 2026. URL <https://arxiv.org/abs/2606.03847v1>.
- [12] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One Step Diffusion via Shortcut Models. In *International Conference on Learning Representations*, volume 2025, pages 34668–34684, 2025. URL [https://proceedings.iclr.cc/paper\\_files/paper/2025/file/559a0998fab1d19b80e7e43a5852401c-Paper-Conference.pdf](https://proceedings.iclr.cc/paper_files/paper/2025/file/559a0998fab1d19b80e7e43a5852401c-Paper-Conference.pdf).
- [13] Yuanchun Guo and Bingyan Liu. Reflex: Real-Time VLA Control through Streaming Inference. *arXiv preprint arXiv:2607.14695v1*, 2026. URL <https://arxiv.org/abs/2607.14695v1>.
- [14] Xiaoshen Han, Haoyu Xiong, Haonan Chen, Chaoqi Liu, Antonio Torralba, Yuke Zhu, and Yilun Du. B-spline Policy: Accelerating Manipulation Policies via B-spline Action Representations. *arXiv preprint arXiv:2607.09648v1*, 2026. URL <https://arxiv.org/abs/2607.09648v1>.
- [15] Jindou Jia, Gen Li, Xiangyu Chen, Tuo An, Yuxuan Hu, Jingliang Li, Xinying Guo, and Jianfei Yang. Action-to-Action Flow Matching. *arXiv preprint arXiv:2602.07322v2*, 2026. URL <https://arxiv.org/abs/2602.07322v2>.

- [16] Hai Jiang, Yixian Zou, Binbin Liang, Boqian Liu, Fanman Meng, and Shuaicheng Liu. FutureRTC: Real-Time Robot Execution with Anticipatory-Conditioned Action Chunking. *arXiv preprint arXiv:2607.24008v1*, 2026. URL <https://arxiv.org/abs/2607.24008v1>.
- [17] Yuhua Jiang, Shuang Cheng, Yan Ding, Feifei Gao, and Biqing Qi. AsyncVLA: Asynchronous Flow Matching for Vision-Language-Action Models. *arXiv preprint arXiv:2511.14148v2*, 2026. URL <https://arxiv.org/abs/2511.14148v2>.
- [18] Dong Jing, Gang Wang, Jiaqi Liu, Weiliang Tang, Zelong Sun, Yunchao Yao, Zhenyu Wei, Yunhui Liu, Zhiwu Lu, and Mingyu Ding. Mixture of Horizons in Action Chunking. *arXiv preprint arXiv:2511.19433v2*, 2026. URL <https://arxiv.org/abs/2511.19433v2>.
- [19] Sinjae Kang, Chanyoung Kim, Kaixin Wang, Li Zhao, and Kimin Lee. WarmPrior: Straightening Flow-Matching Policies with Temporal Priors. *arXiv preprint arXiv:2605.13959v1*, 2026. URL <https://arxiv.org/abs/2605.13959v1>.
- [20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713, 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [21] Sunwoo Kim, Jeongjun Kim, and Joseph Lim. Time Optimal Execution of Action Chunk Policies Beyond Demonstration Speed. In *International Conference on Learning Representations*, volume 2026, pages 154823–154842, 2026. URL [https://proceedings.iclr.cc/paper\\_files/paper/2026/file/fab80bb9d97e9b9ff5c19f91f72838c6-Paper-Conference.pdf](https://proceedings.iclr.cc/paper_files/paper/2026/file/fab80bb9d97e9b9ff5c19f91f72838c6-Paper-Conference.pdf).
- [22] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive Action Chunking at Inference-time for Vision-Language-Action Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20802–20811, 2026.
- [23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. *arXiv preprint arXiv:2210.02747v2*, 2023. URL <https://arxiv.org/abs/2210.02747v2>.
- [24] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional Decoding: Improving Action Chunking via Guided Test-Time Sampling. *arXiv preprint arXiv:2408.17355v4*, 2025. URL <https://arxiv.org/abs/2408.17355v4>.
- [25] Yufeng Liu, Hang Yu, Juntu Zhao, Bocheng Li, Di Zhang, Mingzhu Li, Wenxuan Wu, Yingdong Hu, Junyuan Xie, Junliang Guo, Dequan Wang, and Yang Gao. Learning Native Continuation for Action Chunking Flow Policies. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.058.
- [26] Yushan Liu, Peibo Sun, Xintao Chao, Zhenyang Yang, Yifan Xie, Lingfeng Zhang, Shoujie Li, Chenyu Tang, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. CheckVLA: Execution-Time Verification with Action-Conditioned World Model for Long-Horizon Mobile Manipulation. *arXiv preprint arXiv:2607.26789v1*, 2026. URL <https://arxiv.org/abs/2607.26789v1>.
- [27] Yuxiang Lu, Zhe Liu, Xianzhe Fan, Zhenya Yang, Jinghua Hou, Junyi Li, Kaixin Ding, and Hengshuang Zhao. FASTER: Rethinking Real-Time Flow VLAs. *arXiv preprint arXiv:2603.19199v3*, 2026. URL <https://arxiv.org/abs/2603.19199v3>.
- [28] Wuyang Luan, Junhui Li, Weiguang Zhao, Wenjian Zhang, Tieru Wu, and Rui Ma. SnapFlow: One-Step Action Generation for Flow-Matching VLAs via Progressive Self-Distillation. *arXiv preprint arXiv:2604.05656v1*, 2026. URL <https://arxiv.org/abs/2604.05656v1>.
- [29] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running VLAs at Real-time Speed. *arXiv preprint arXiv:2510.26742v1*, 2025. URL <https://arxiv.org/abs/2510.26742v1>.
- [30] Rhea Malhotra, Yuejiang Liu, and Chelsea Finn. Self-Guided Action Diffusion. *arXiv preprint arXiv:2508.12189v1*, 2025. URL <https://arxiv.org/abs/2508.12189v1>.

- [31] Junnan Nie, Jiayi Li, Chenghao Liu, Junyi Lao, Jiachen Zhang, Tianle Zhang, Liang Lin, and Songfang Huang. PACE: Phase-Aware Chunk Execution for Robot Policies with Action Chunking. *arXiv preprint arXiv:2606.00537v3*, 2026. URL <https://arxiv.org/abs/2606.00537v3>.
- [32] Jiahui Niu, Kefan Gu, Yucheng Zhao, Shengwen Liang, Tiancai Wang, Xing Hu, Ying Wang, and Huawei Li. Realtime-VLA FLASH: Speculative Inference Framework for Diffusion-based VLAs. *arXiv preprint arXiv:2605.13778v1*, 2026. URL <https://arxiv.org/abs/2605.13778v1>.
- [33] Sungjae Park and Shubham Tulsiani.  $\pi\mathbf{R}^2$ : Reactive Real-time Flow Policies. *arXiv preprint arXiv:2607.26055v1*, 2026. URL <https://arxiv.org/abs/2607.26055v1>.
- [34] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient Action Tokenization for Vision-Language-Action Models. *arXiv preprint arXiv:2501.09747v1*, 2025. URL <https://arxiv.org/abs/2501.09747v1>.
- [35] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Hao-huan Wang, Lili Yu, and Ury Zhilinsky.  $\pi_{0.5}$ : a Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054v1*, 2025. URL <https://arxiv.org/abs/2504.16054v1>.
- [36] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency Policy: Accelerated Visuomotor Policies via Consistency Distillation. *arXiv preprint arXiv:2405.07503v2*, 2024. URL <https://arxiv.org/abs/2405.07503v2>.
- [37] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. Leave No Observation Behind: Real-time Correction for VLA Action Chunks. *arXiv preprint arXiv:2509.23224v1*, 2025. URL <https://arxiv.org/abs/2509.23224v1>.
- [38] Yiran Shi, Dongqi Guo, Tianchen Zhao, Feng Gao, Liangzhi Shi, Chao Yu, ZhiJian Mo, Qihua Xiao, Xiaoshuai Peng, Qingmin Liao, and Yu Wang. StreamingVLA: Streaming Vision-Language-Action Model with Action Flow Matching and Adaptive Early Observation. *arXiv preprint arXiv:2603.28565v1*, 2026. URL <https://arxiv.org/abs/2603.28565v1>.
- [39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844v1*, 2025. URL <https://arxiv.org/abs/2506.01844v1>.
- [40] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhi-jian Liu, and Song Han. VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. *arXiv preprint arXiv:2512.01031v2*, 2026. URL <https://arxiv.org/abs/2512.01031v2>.
- [41] Mengze Tian, Yiming Li, Sichao Liu, Auke Ijspeert, and Sylvain Calinon. Spline Policy: A Structured Representation for Robot Policies. *arXiv preprint arXiv:2606.07386v2*, 2026. URL <https://arxiv.org/abs/2606.07386v2>.
- [42] Haoxuan Wang, Gengyu Zhang, Yan Yan, Ramana Rao Kompella, and Gaowen Liu. VLA Knows Its Limits: Adaptive Execution Horizons for Robot Policies. *arXiv preprint arXiv:2602.21445v2*, 2026. URL <https://arxiv.org/abs/2602.21445v2>.
- [43] Haoxuan Wang, Gengyu Zhang, Yan Yan, Yuzhang Shang, Ramana Kompella, and Gaowen Liu. Real-Time Robot Execution with Masked Action Chunking. In *International Conference on Learning Representations*, volume 2026, pages 120161–120179, 2026. URL [https://proceedings.iclr.cc/paper\\_files/paper/2026/file/c35834443b7881e782e52b3519fe27c7-Paper-Conference.pdf](https://proceedings.iclr.cc/paper_files/paper/2026/file/c35834443b7881e782e52b3519fe27c7-Paper-Conference.pdf).
- [44] Zhendong Wang, Max Li, Ajay Mandlikar, Zhenjia Xu, Jiaojiao Fan, Yashraj Narang, Linxi Fan, Yuke Zhu, Yogesh Balaji, Mingyuan Zhou, Ming-Yu Liu, and Yu Zeng. One-Step Diffusion Policy: Fast Visuomotor Policies via Diffusion Distillation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63399–63416, 2025. URL <https://proceedings.mlr.press/v267/wang25ba.html>.

- [45] Han Xue, Jieji Ren, Wendi Chen, Gu Zhang, Yuan Fang, Guoying Gu, Huazhe Xu, and Cewu Lu. Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. *arXiv preprint arXiv:2503.02881v3*, 2025. URL <https://arxiv.org/abs/2503.02881v3>.
- [46] Chen Yang, Yucheng Hu, Yunchao Ma, Yunhuan Yang, Jing Tan, and Haoqiang Fan. Realtime-VLA V2: Learning to Run VLAs Fast, Smooth, and Accurate. *arXiv preprint arXiv:2603.26360v1*, 2026. URL <https://arxiv.org/abs/2603.26360v1>.
- [47] Fan Yang, Peiguang Jing, Kaihua Qu, Ningyuan Zhao, and Yuting Su. ABPolicy: Asynchronous B-Spline Flow Policy for Real-Time and Smooth Robotic Manipulation. *arXiv preprint arXiv:2602.23901v1*, 2026. URL <https://arxiv.org/abs/2602.23901v1>.
- [48] Zhao Yang, Yinan Shi, Mingyuan Yao, Wenyao Xue, Yawei Jueluo, and Longjun Liu. ChunkFlow: Towards Continuity-Consistent Chunked Policy Learning. *arXiv preprint arXiv:2607.12992v1*, 2026. URL <https://arxiv.org/abs/2607.12992v1>.
- [49] Dijia Zhan, Xuemiao Xu, Jinyi Li, and Jie Tang. SEAM: Smooth Execution of Action-Chunked Motion for Vision-Language-Action Policies. *arXiv preprint arXiv:2607.04609v1*, 2026. URL <https://arxiv.org/abs/2607.04609v1>.
- [50] Jiyao Zhang, Zimu Han, Junhan Wang, Xionghao Wu, Shihong Lin, Jinzhou Li, Hongwei Fan, Ruihai Wu, Dongjiang Li, and Hao Dong. HiPolicy: Hierarchical Multi-Frequency Action Chunking for Policy Learning. *arXiv preprint arXiv:2604.06067v1*, 2026. URL <https://arxiv.org/abs/2604.06067v1>.
- [51] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.016.
- [52] Hongyi Zhou, Weiran Liao, Xi Huang, Yucheng Tang, Fabian Otto, Xiaogang Jia, Xinkai Jiang, Simon Hilber, Ge Li, Qian Wang, Ömer Yağmurlu, Nils Blank, Moritz Reuss, and Rudolf Lioutikov. BEAST: Efficient Tokenization of B-Splines Encoded Action Sequences for Imitation Learning. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 172934–172959, 2025. doi: 10.52202/085713-5755. URL [https://proceedings.neurips.cc/paper\\_files/paper/2025/file/fc75cfcecl6170f2b54d00283c739928-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2025/file/fc75cfcecl6170f2b54d00283c739928-Paper-Conference.pdf).
- [53] Lifeng Zhuo, Wendi Chen, Han Xue, Shirun Tang, Jun Lv, Cewu Lu, and Chuan Wen. FA-RDP: A Frequency-Adaptive Reactive Diffusion Policy for Contact-Rich Manipulation. *arXiv preprint arXiv:2607.28596v1*, 2026. URL <https://arxiv.org/abs/2607.28596v1>.
- [54] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.

# Appendix

## A Task Performance Across Garment Conditions

This appendix partitions the 180 physical trials in the main text by three garment conditions. For each condition, it reports successful trials, mean completion time, and successful-task throughput. The analysis complements the aggregate results by showing how each execution method behaves across manipulation objects.

**Table S1. Task-level results across garment conditions.** Each row contains ten physical trials, and throughput is the number of successful tasks completed per hour.

| Method             | Garment condition | Successes | Mean time (s) | Throughput ( $\text{h}^{-1}$ ) |
|--------------------|-------------------|-----------|---------------|--------------------------------|
| VLASH              | Small purple      | 10/10     | 56.4          | 63.83                          |
|                    | Large pink        | 9/10      | 99.2          | 32.66                          |
|                    | Medium red        | 9/10      | 77.6          | 41.75                          |
| Training-time RTC  | Small purple      | 8/10      | 116.5         | 24.72                          |
|                    | Large pink        | 7/10      | 124.4         | 20.26                          |
|                    | Medium red        | 4/10      | 147.2         | 9.78                           |
| Legato             | Small purple      | 10/10     | 72.47         | 49.68                          |
|                    | Large pink        | 10/10     | 69.2          | 52.02                          |
|                    | Medium red        | 9/10      | 79.0          | 41.01                          |
| Temporal Smoothing | Small purple      | 8/10      | 84.1          | 34.24                          |
|                    | Large pink        | 9/10      | 103.2         | 31.40                          |
|                    | Medium red        | 6/10      | 94.5          | 22.86                          |
| Naive Asynchronous | Small purple      | 6/10      | 134.4         | 16.07                          |
|                    | Large pink        | 5/10      | 144.5         | 12.46                          |
|                    | Medium red        | 8/10      | 93.5          | 30.80                          |
| Inference-time RTC | Small purple      | 7/10      | 149.5         | 16.86                          |
|                    | Large pink        | 7/10      | 129.5         | 19.46                          |
|                    | Medium red        | 4/10      | 122.2         | 11.78                          |

Table S1 shows that Legato and VLASH complete 9/10–10/10 tasks under all three garment conditions, leading overall in success rate and throughput. Legato maintains throughput of at least  $41.01 \text{ h}^{-1}$ , while VLASH achieves the highest value in the table,  $63.83 \text{ h}^{-1}$ , on the small purple garment. Temporal Smoothing completes 6/10–9/10 tasks with throughput of  $22.86$ – $34.24 \text{ h}^{-1}$ , performing well among training-free methods. Both RTC methods complete only 4/10 tasks on the medium red garment. These results indicate differences in sensitivity to garment size and appearance.

## B Training Configuration

Table S2 summarizes the key settings needed to reproduce model training. Only settings that directly affect training and reproducibility are retained; platform-management parameters such as logging frequency, checkpoint intervals, and runtime paths are omitted.

**Table S2. Training configuration for two-stage non-uniform denoising.**

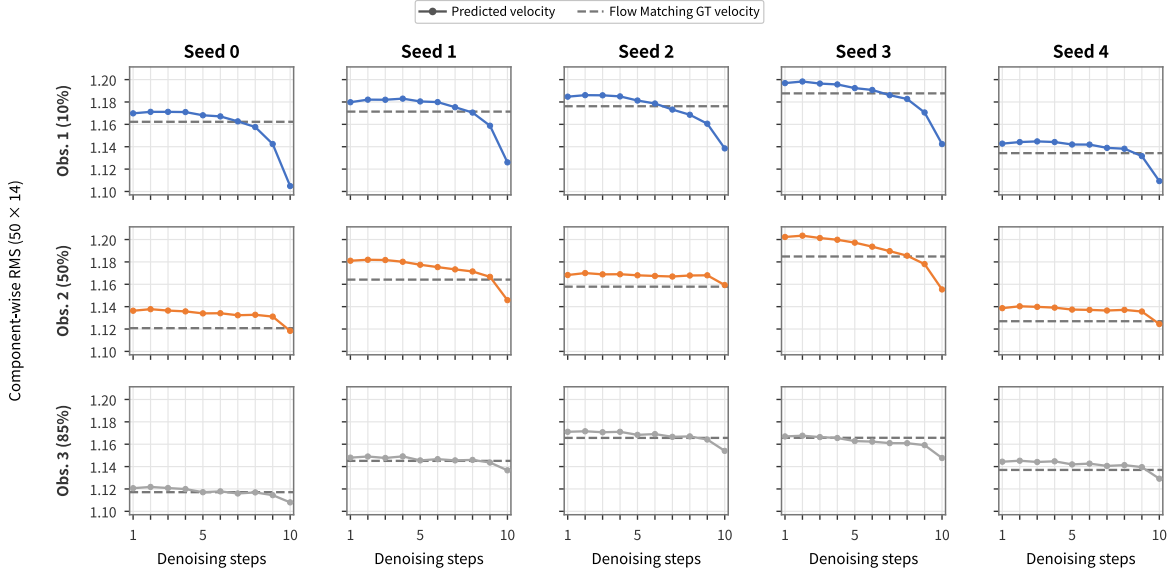
| Hyperparameter                       | Value                                                                                                                     |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Training dataset                     | 3,048 teleoperated demonstration trajectories                                                                             |
| Training steps                       | 30,000                                                                                                                    |
| Batch size                           | 16                                                                                                                        |
| Optimizer                            | AdamW, $\beta_1 = 0.9$ , $\beta_2 = 0.95$ , $\epsilon = 10^{-8}$                                                          |
| Learning-rate schedule               | Peak learning rate $2.5 \times 10^{-5}$ ; linear warmup for 1,000 steps, followed by cosine decay to $2.5 \times 10^{-6}$ |
| Regularization and gradient clipping | Weight decay 0.01; maximum gradient norm 1.0                                                                              |

## C Scale Consistency of the Flow Matching Velocity Field

Figure 3 shows stage-dependent differences in velocity magnitude and directional correction during standard 10-NFE inference. To assess whether predicted velocities match the scale of the training targets, this appendix uses the component-wise RMS defined in Eq. (3) to compare predicted and Flow Matching ground-truth (GT) velocities. This complements the main-text analysis from the perspective of supervision scale.

### C.1 Scale Comparison with Flow Matching GT Velocity

We use the local velocity target  $v^*$  defined in Section 3 as the Flow Matching GT velocity and compute its component-wise RMS in the valid 14-dimensional space. Five fixed noise seeds at each of three progress positions (0.10, 0.50, and 0.85) in the same representative validation sequence yield 15 inference trajectories. For a fixed action–noise pair,  $v^*$  does not change across denoising steps, so its RMS appears as a horizontal dashed line in Fig. S1. This velocity is a sample-level supervision target, not the magnitude of the dataset action itself.



**Figure S1. Component-wise RMS comparison between predicted and Flow Matching GT velocity vectors in a representative validation sequence.** Colored solid curves show predicted velocity RMS; black horizontal dashed lines show the corresponding GT velocity RMS.

Across 15 trajectories and 10 denoising steps, the mean absolute relative deviation is 0.775%, indicating comparable predicted and GT velocity RMS throughout integration. At step 10, 14 trajectories have predicted velocity RMS slightly below the corresponding GT value. The mean signed relative deviation is  $-1.813\%$ , with individual values ranging from  $-4.941\%$  to  $0.127\%$ . These results indicate that terminal updates mainly involve small velocity adjustments rather than reconstruction of the overall velocity magnitude. This agrees with the broadly stable early and intermediate magnitudes and stronger terminal directional corrections in Fig. 3. The supervision-scale comparison complements the stage analysis and supports long-interval coarse generation followed by short-interval terminal refinement.

## D Offline Action Accuracy of Two-Stage Non-Uniform Denoising

This appendix supplements offline sequence II in the main text with frame-wise joint trajectories, error distributions, and bimanual TCP pose errors.

### D.1 Comparison and Evaluation Protocol

We use the model comparison and input protocol in Section 5.2. The following details concern only frame-wise processing and metric computation for offline sequence II.

The fixed validation sequence contains 376 frames over 12.5 s. Each frame independently uses its recorded observation. Following the 14-dimensional action convention in the main text, the model’s  $50 \times 32$  output is aligned with the recorded action as `action_chunk[0, :14]` versus `action[t, :14]`. Errors for the 12 rotational joints are reported in rad, while the two grippers are reported separately in dataset command units. Predicted state is not recursively passed to the next frame.

Bimanual TCP poses are computed by forward kinematics using the same robot Unified Robot Description Format (URDF). For each arm, TCP position is the midpoint between the origins of the two fingertip links; orientation uses the corresponding sixth-axis end-link frame. For frame  $t$  and arm  $a \in \{L, R\}$ , let  $(\hat{\mathbf{p}}_{t,a}, \hat{\mathbf{R}}_{t,a})$  denote the predicted pose and  $(\mathbf{p}_{t,a}^*, \mathbf{R}_{t,a}^*)$  the reference pose obtained from the recorded action. Translation and rotation errors are

$$e_{t,a}^{\text{trans}} = \|\hat{\mathbf{p}}_{t,a} - \mathbf{p}_{t,a}^*\|_2, \quad (17)$$

$$e_{t,a}^{\text{rot}} = \arccos \left[ \text{clip} \left( \frac{\text{tr}((\mathbf{R}_{t,a}^*)^\top \hat{\mathbf{R}}_{t,a}) - 1}{2}, -1, 1 \right) \right] \frac{180}{\pi}. \quad (18)$$

The units of  $e_{t,a}^{\text{trans}}$  and  $e_{t,a}^{\text{rot}}$  are mm and degrees, respectively. For  $T$  evaluated frames, table entries are arithmetic means:

$$E_a^{(\cdot)} = \frac{1}{T} \sum_{t=1}^T e_{t,a}^{(\cdot)}. \quad (19)$$

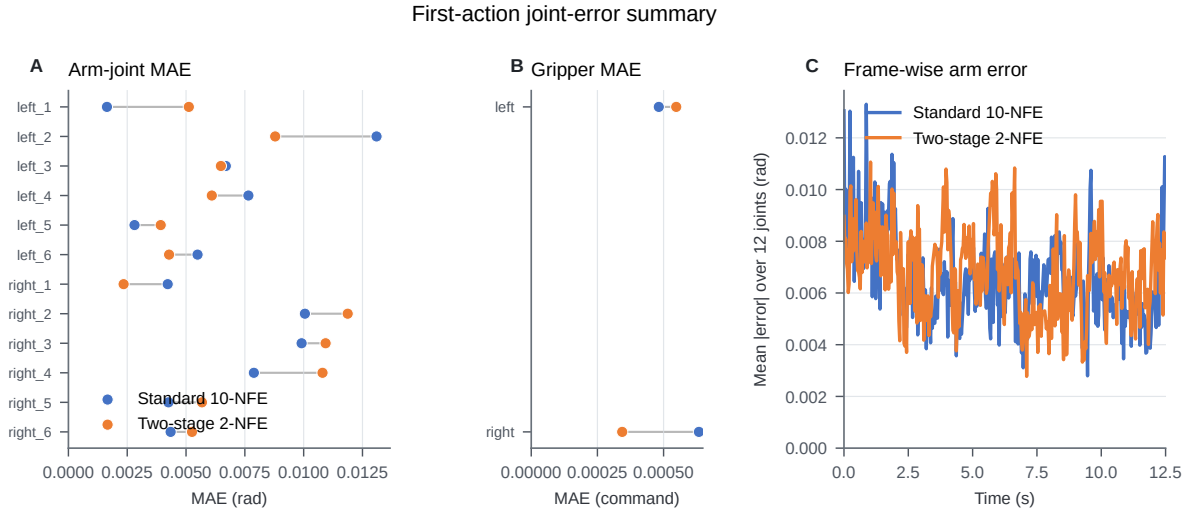
After excluding first-frame JIT, the sequence contributes 375 samples. Mean inference times are 61.571 ms for standard 10-NFE and 21.923 ms for two-stage 2-NFE, corresponding to a  $2.808\times$  speedup and a 64.39% reduction in time.

**Table S3. Supplementary action-error metrics on the fixed validation sequence.**

| Metric                                             | Standard $\pi_{0.5}$ ,<br>10 NFE | Two-stage,<br>2 NFE |
|----------------------------------------------------|----------------------------------|---------------------|
| 12-joint 95th-percentile absolute error (P95, rad) | 0.019209                         | 0.018704            |
| 12-joint maximum absolute error (rad)              | 0.057459                         | 0.052449            |
| Left/right gripper MAE (command unit)              | 0.000482 / 0.000634              | 0.000548 / 0.000344 |
| Left/right TCP rotation MAE ( $^\circ$ )           | 1.047 / 0.902                    | 0.911 / 1.169       |

## D.2 Per-Joint Error and Frame-Wise Variation

Figure S2 reports per-joint MAE, left and right gripper MAE, and frame-wise mean absolute error over the 12 joints. The two-stage model has P95 and maximum absolute errors of 0.018704 rad and 0.052449 rad, respectively, both lower than the standard model. The panels show differences for individual joints and grippers.

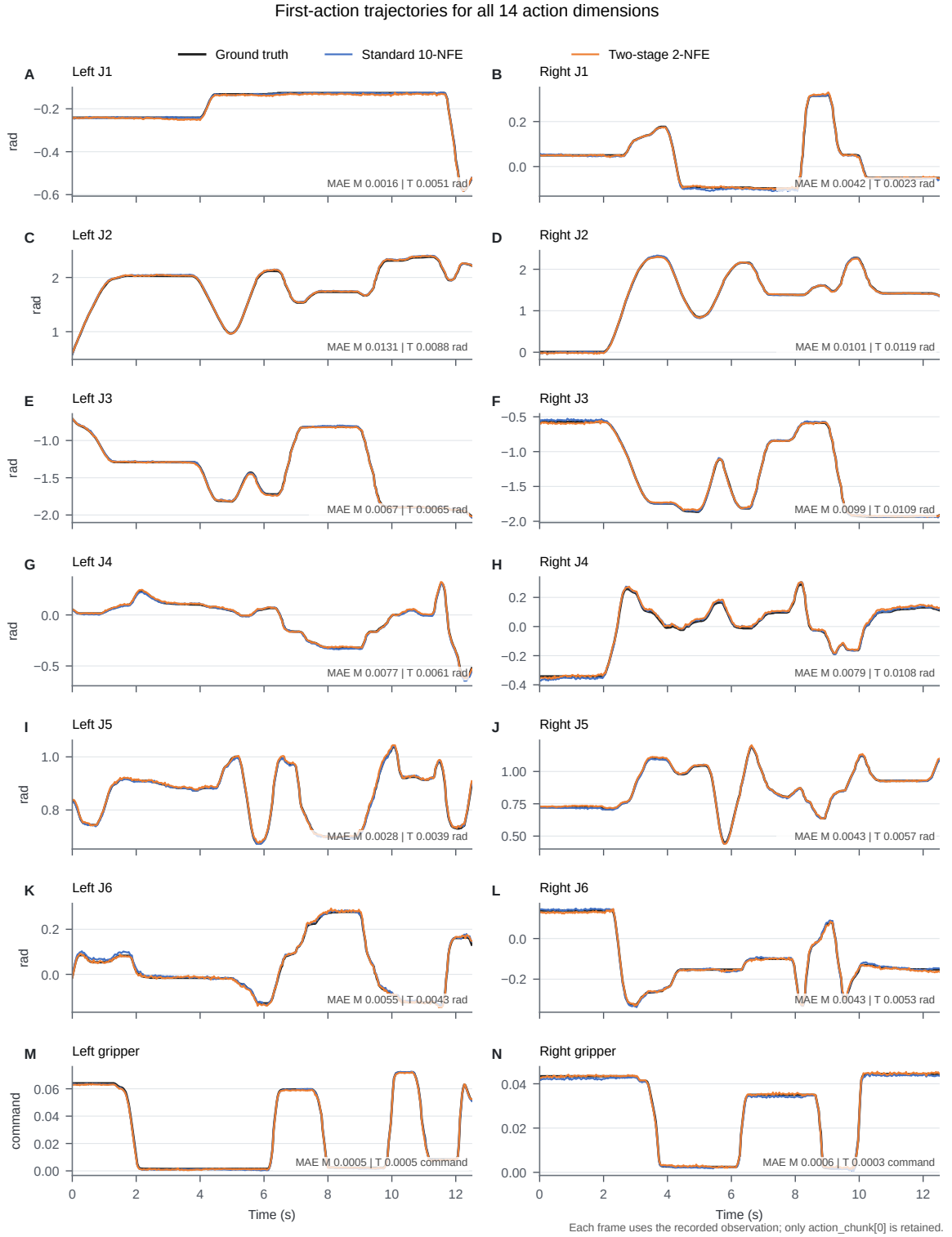


**Figure S2. Per-joint action errors on the fixed validation sequence.** (A) MAE for 12 rotational joints. (B) Left and right gripper-command MAE. (C) Frame-wise mean absolute error over the 12 rotational joints. Blue denotes the standard 10-NFE model, and orange denotes the two-stage 2-NFE model.

## D.3 Fourteen-Dimensional Per-Joint Action Trajectories

Figure S3 shows 14-dimensional recorded and predicted action trajectories. Black curves denote recorded actions; blue and orange curves denote predictions from the standard 10-NFE and two-stage 2-NFE models. The left and right columns show six rotational joints and one gripper command for the respective arms. M/T indicates the per-dimension MAE of the standard

and two-stage models, respectively. Across joints and grippers, the two-stage predictions broadly retain the main trends of the recorded actions.

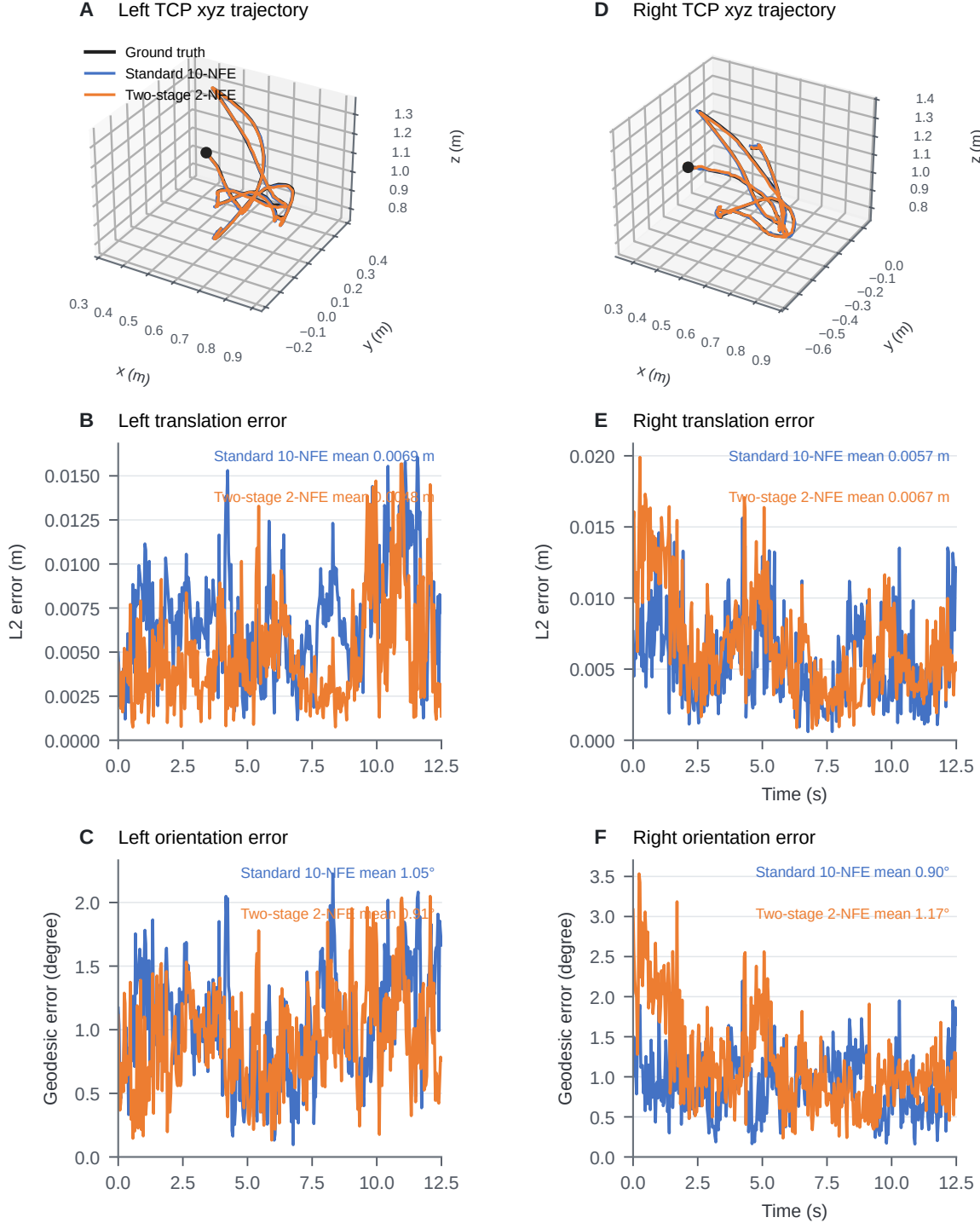


**Figure S3. Per-joint comparison.**

## D.4 Bimanual TCP Trajectories and Pose Errors

As shown in Fig. S4, two-stage denoising reduces left-arm TCP rotation MAE from  $1.047^\circ$  to  $0.911^\circ$ , while increasing the right-arm value from  $0.902^\circ$  to  $1.169^\circ$ . The three-dimensional trajectories and frame-wise translation errors further show temporal differences between the models.

Bilateral TCP trajectories and pose errors



Translation is Euclidean TCP error; orientation is SO(3) geodesic angle.

**Figure S4. Bimanual TCP trajectories and pose errors on the fixed validation sequence.** (A,D) Three-dimensional left and right TCP trajectories. (B,E) Euclidean translation error. (C,F) SO(3) geodesic orientation error. Black denotes the TCP trajectory obtained by applying URDF forward kinematics to the recorded action. Blue and orange denote the standard 10-NFE and two-stage 2-NFE models.

## E Additional Execution Figures

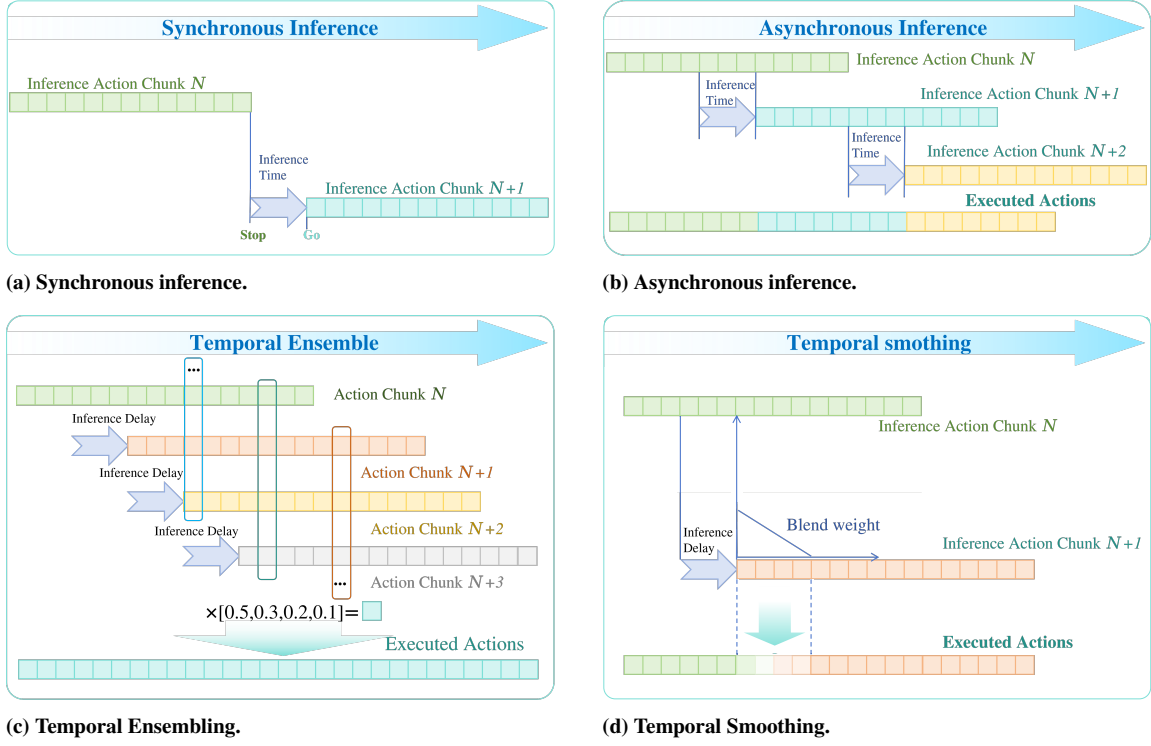


Figure S5. Reference inference and execution modes.

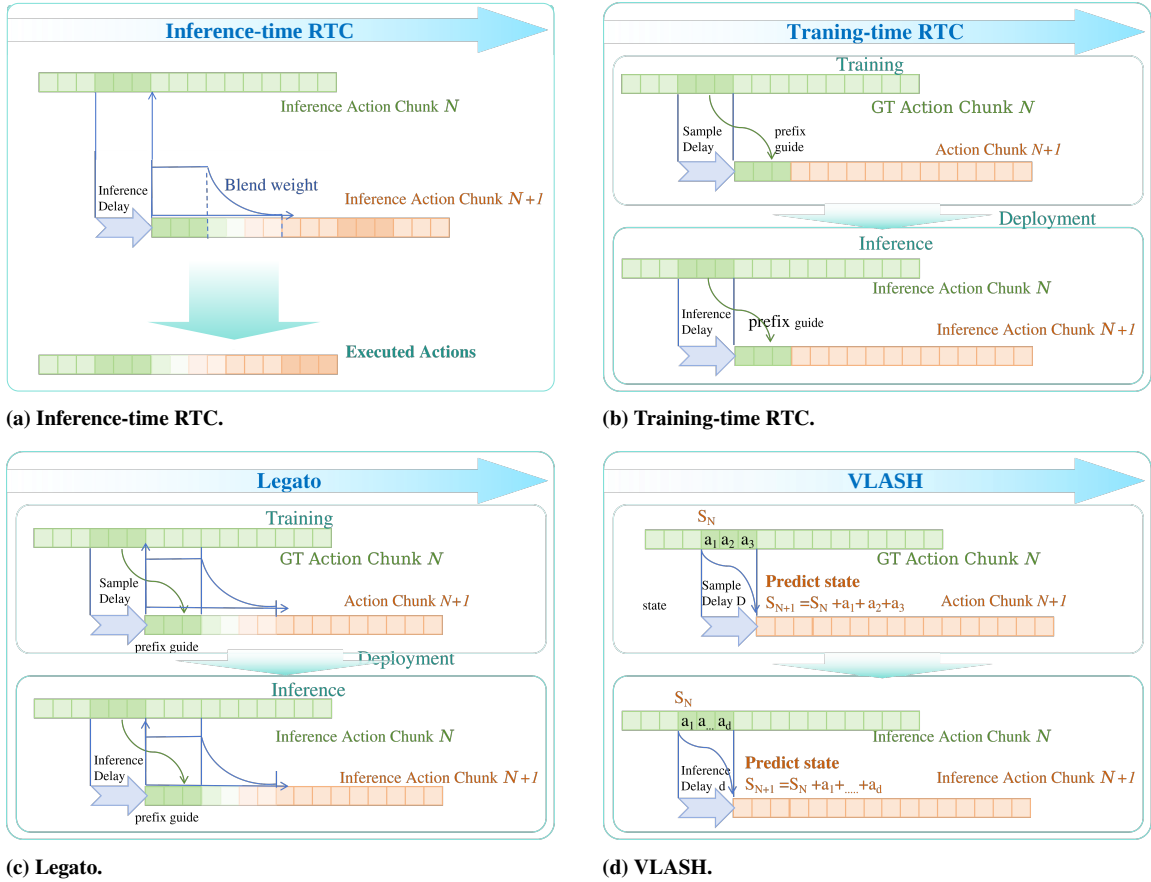
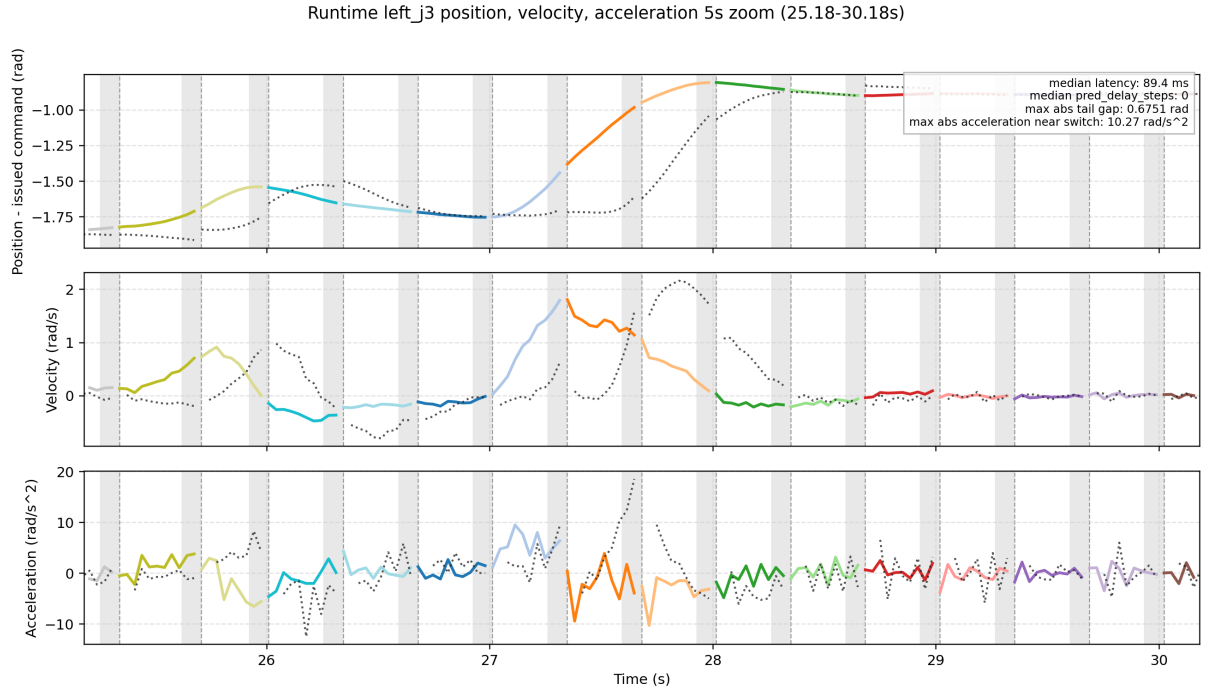
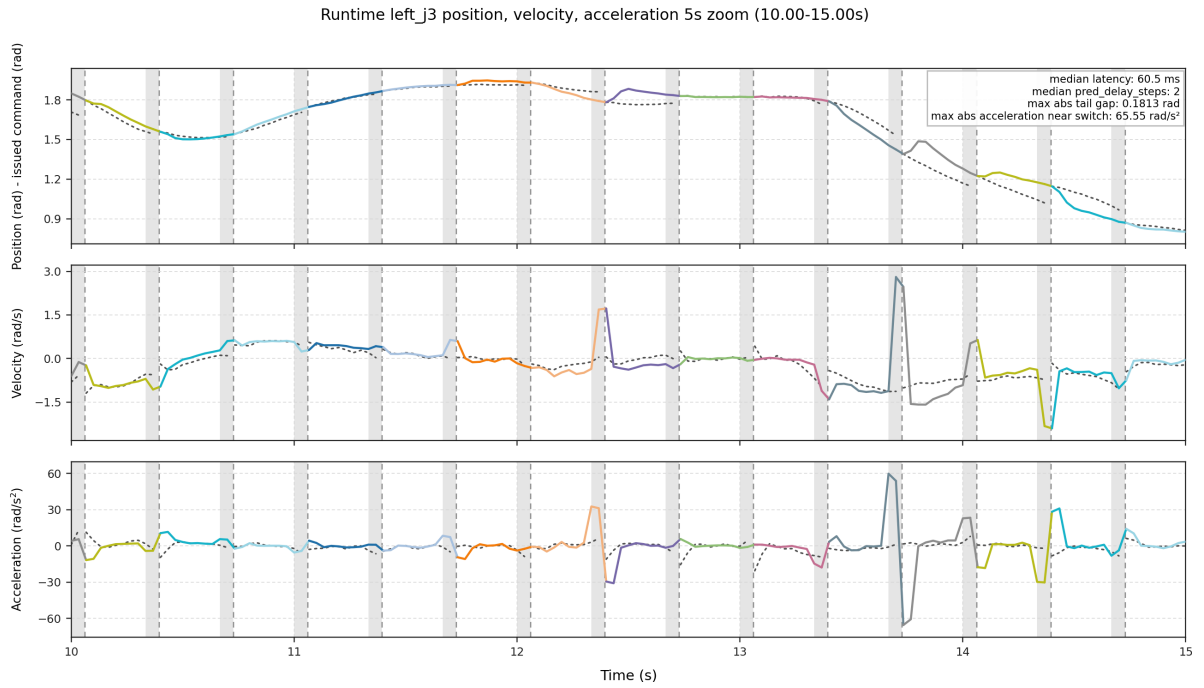


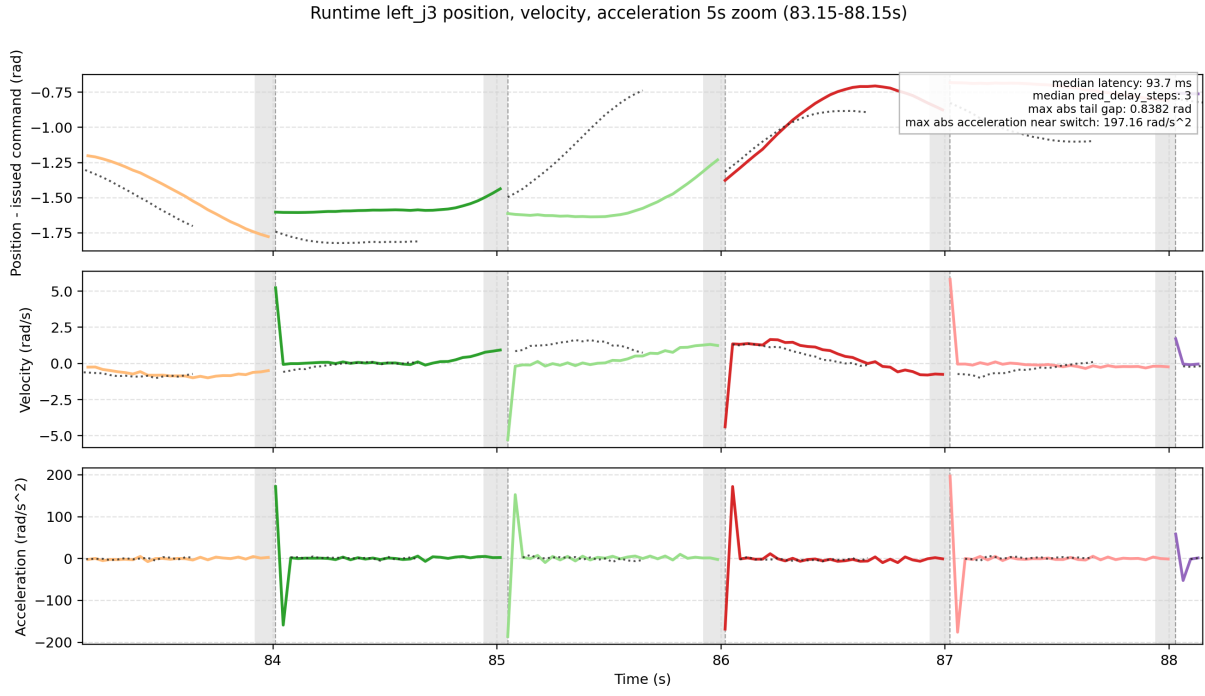
Figure S6. Representative delay-aware execution methods.



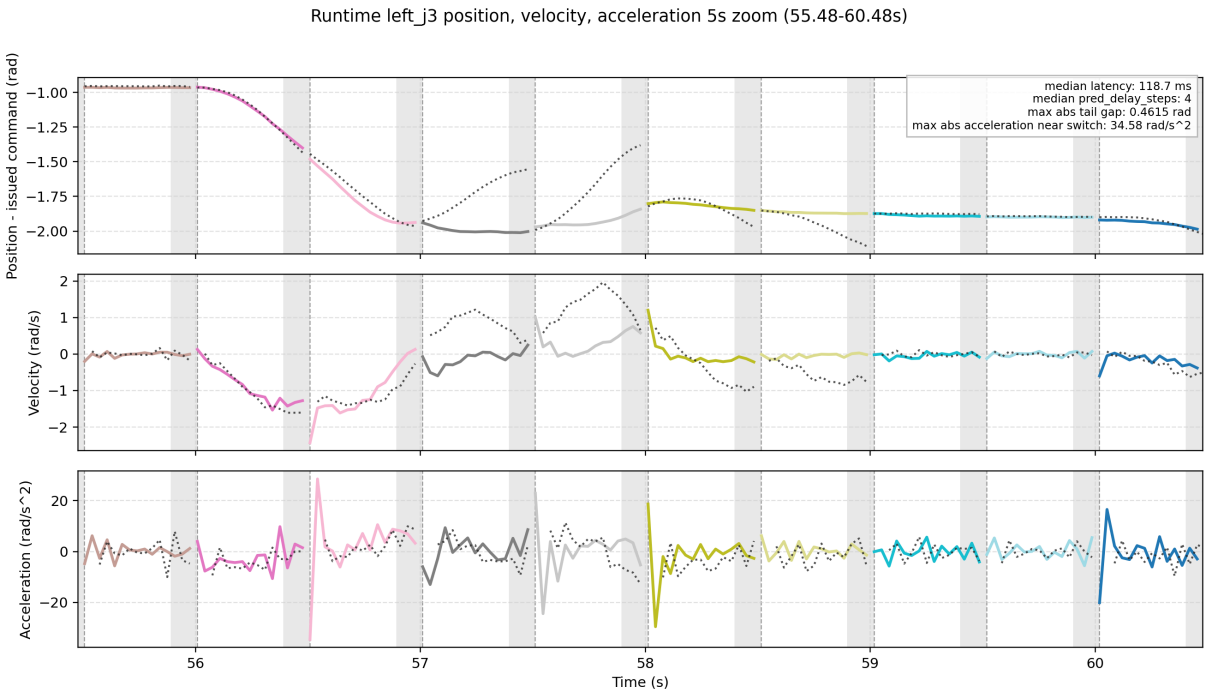
**Figure S7. Local left\_j3 trajectory for Temporal Smoothing.**



**Figure S8. Local left\_j3 trajectory for Inference-time RTC.**



**Figure S9. Local left\_j3 trajectory for Training-time RTC.**



**Figure S10. Local left\_j3 trajectory for VLASH.**